AIAA 3102 · Python Programming with AI

深度学习 II

深度学习 II

Deep Learning II

中英对照 · 双页阅读

AIAA 3102 （2026-27 秋季）

Python 编程与人工智能

深度学习

Chengwei QIN@AI Thrust

AIAA 3102 (2026-27 Fall)

Python Programming with Artificial Intelligence

Deep Learning

Chengwei QIN@AI Thrust

今日提纲

深度模型架构

- 多层感知机（MLP）
- 卷积神经网络（CNN）
- 序列建模与 RNN
- 语言建模与 Transformer

训练策略

- 正则化
- 归一化
- 权重初始化与优化

Today's Outline

Deep Model Architecture

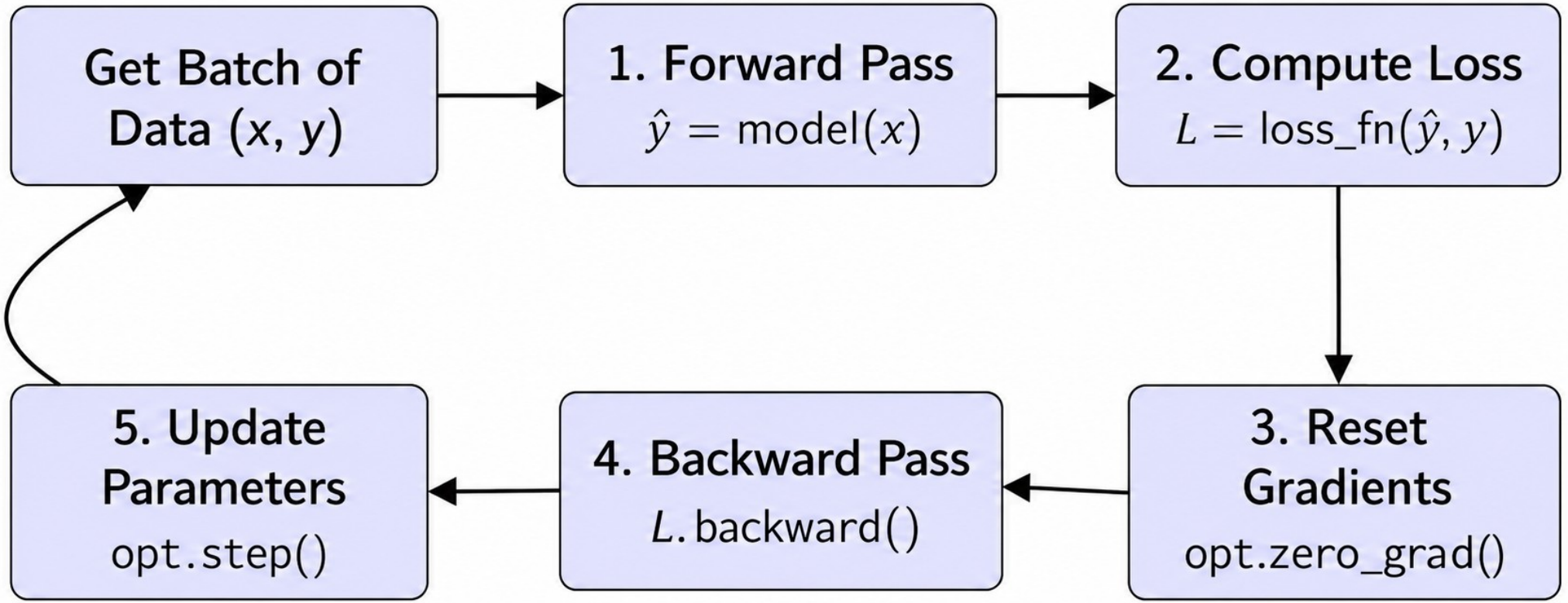
- Multi-Layer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Sequence Modeling and RNN
- Language Modeling and Transformer

Training Strategy

- Regularization
- Normalization
- Weight Initialization & Optimization

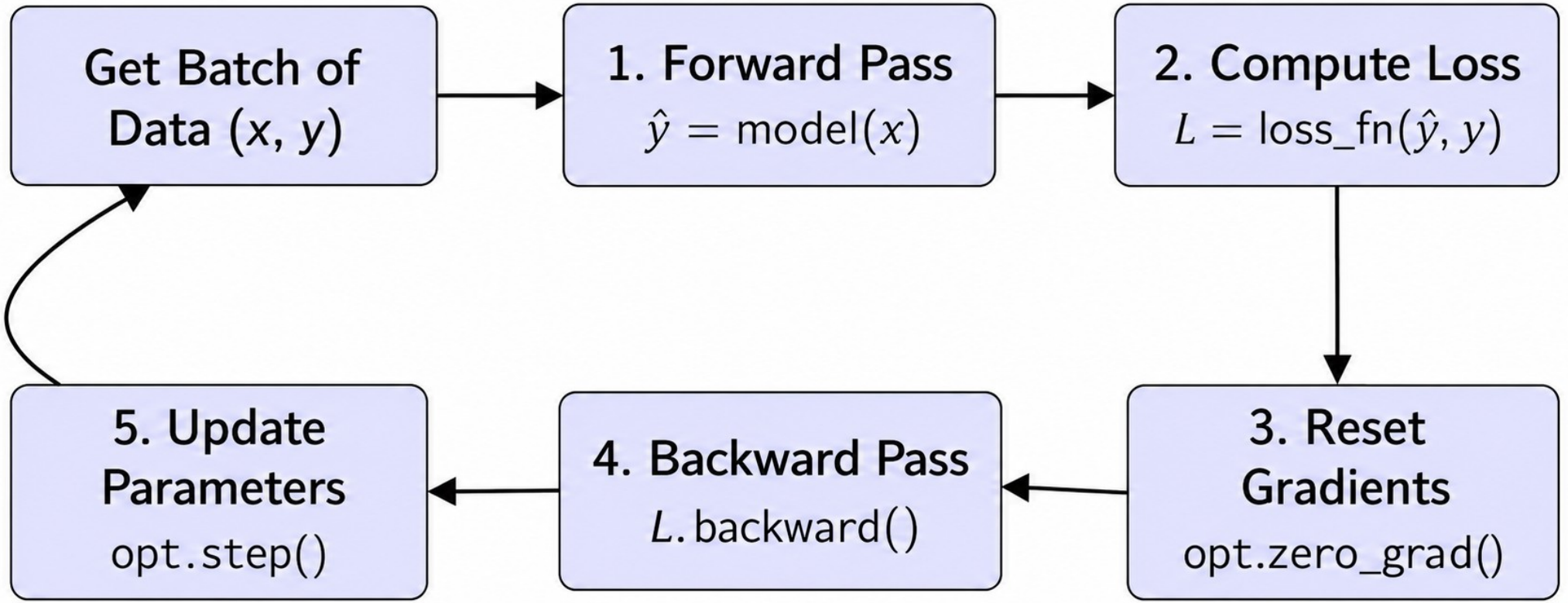
回顾：训练循环

为所有权重（w）和偏置（b）找到“合适”的值



Recap: The Training Loop

Find the "right" values for all the weights (w) and biases (b)



优秀深度学习程序的配方

两个关键问题

- **如何设计模型架构?**
  选择合适的层，并在代码中合理组织。

- **如何训练模型?**
  选择合适的损失函数、初始化、优化器和正则化

The Recipe for a Good Deep Learning Program

Two Key Questions

- **How do we design the model architecture?**
  Choose suitable layers and organize them properly in code.

- **How do we train the model?**
  Choose proper loss functions, initialization, optimizers, regularization



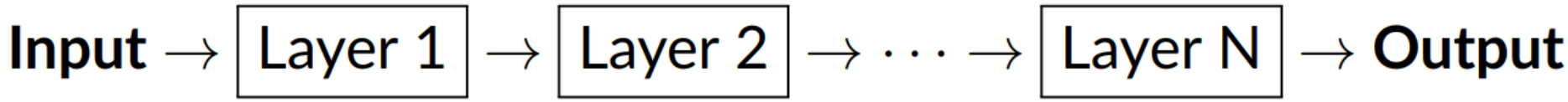
模型架构



Model Architecture

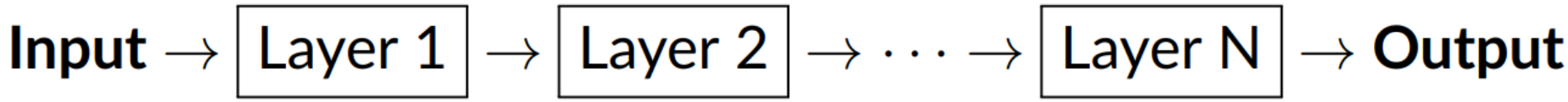
深度模型

“深”指模型的深度，即层数。
多层结构使模型能从数据中自动学习复杂概念的层次结构。



Deep Model

"Deep" refers to the depth of the model, i.e., the number of layers. Multiple layers allow the model to learn a hierarchy of complex concepts automatically from data.



深度模型架构设计

没有放之四海而皆准的方案，也没有万能捷径。

为任务选择合适的工具：

- MLP 用于简单的结构化数据
- CNN 用于图像
- RNN 用于序列数据
- Transformer 用于语言建模

AI 社区已开发出许多可复用的构建模块。

理解它们的设计逻辑有助于为实际项目定制模型。

接下来介绍 PyTorch 中几种常用的构建模块。

Deep Model Architecture Design

There is no one-size-fits-all solution and no universal shortcut.

Choose the right tool for the right task:

- MLPs for simple structured data
- CNNs for images
- RNNs for sequential data
- Transformers for language modeling

The AI community has developed many reusable building blocks.

Understanding their design logic helps us customize models for real-world projects.

Next, we introduce several commonly used building blocks in PyTorch.

概览

- 多层感知机（MLP）
- 卷积神经网络（CNN）
- 序列建模与 RNN
- 语言建模与 Transformer

Overview

- Multi-Layer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Sequence Modeling and RNN
- Language Modeling and Transformer



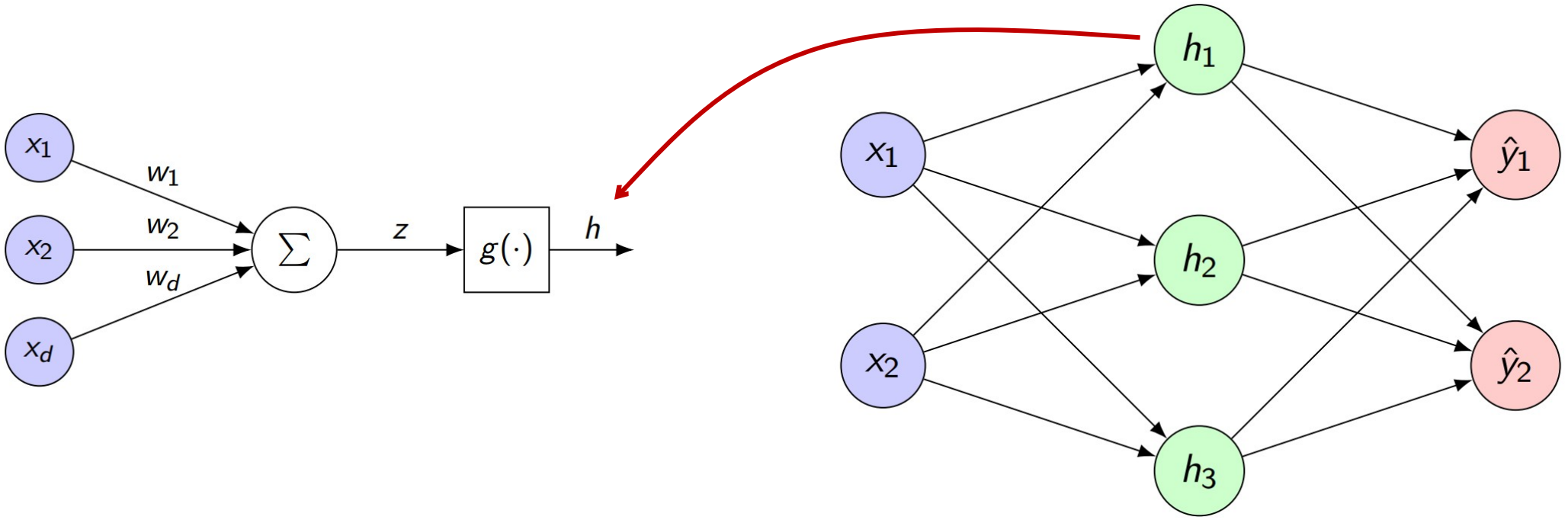
MLP



MLP

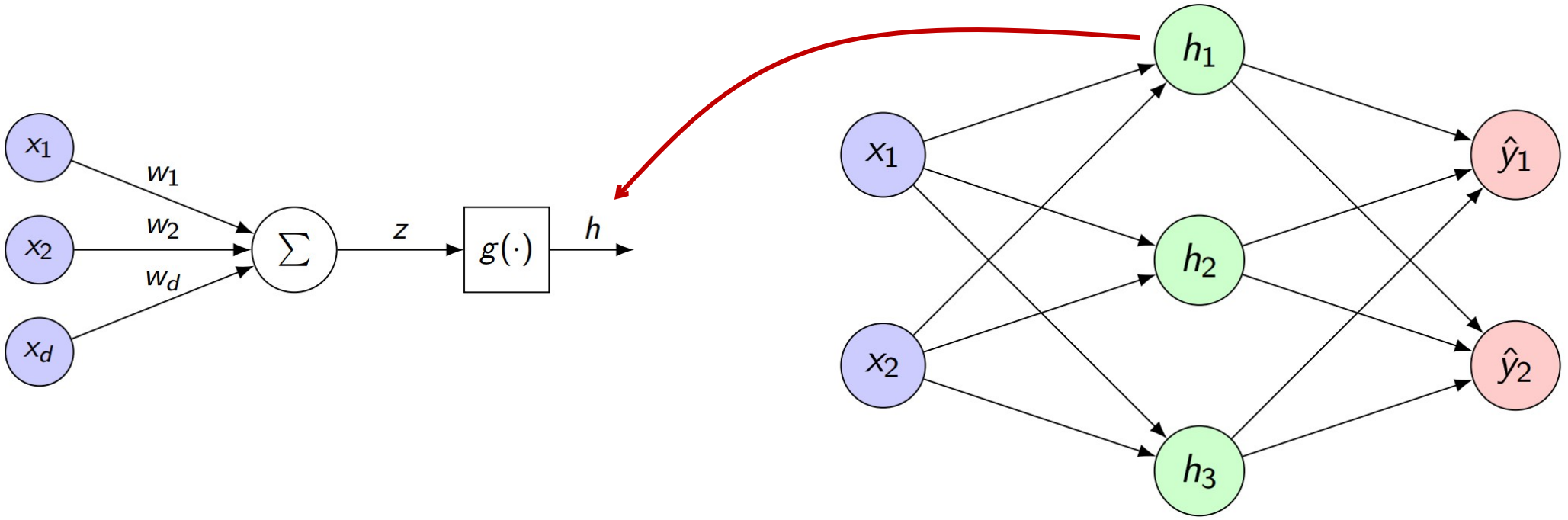
多层感知机（MLP）

- 神经元先计算输入的加权组合，再施加非线性变换，相当于一个简单的特征检测器。
- 有 M 个输入、 N 个输出的层称为**全连接层或线性层**。
- 即矩阵乘法后接一个**激活函数**。
- 输入层 → 隐藏层 → 输出层



Multi-Layer Perceptron (MLP)

- A neuron computes a weighted combination of inputs and then applies a nonlinearity. It acts as a simple feature detector.
- A layer with M inputs and N outputs is a **fully-connected or linear layer**.
- A matrix multiplication followed by an **activation**.
- Input Layer → Hidden Layer(s) → Output Layer



激活函数的作用

多个线性层堆叠起来仍然只是一个线性层。
网络无法学习复杂的非线性模式。
必须在线性层之后施加非线性激活函数。

Layer 1: $y_1 = W_1 x + b_1$

Layer 2: $y_2 = W_2 y_1 + b_2$

把第 1 层代入第 2 层：

$$y_2 = W_2(W_1 x + b_1) + b_2$$

$$y_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$$

$$y_2 = W_{new} x + b_{new}$$

The Role of Activation Functions

A stack of linear layers is one single linear layer.

The network cannot learn complex, non-linear patterns.

We must apply a non-linear activation function after the linear layers.

Layer 1: $y_1 = W_1x + b_1$

Layer 2: $y_2 = W_2y_1 + b_2$

Substitute Layer 1 into Layer 2:

$$y_2 = W_2(W_1x + b_1) + b_2$$

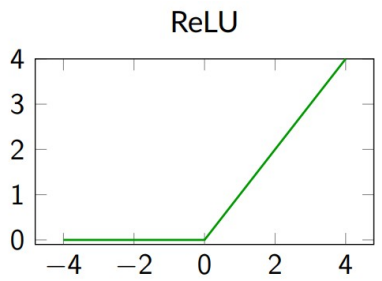
$$y_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

$$y_2 = W_{new}x + b_{new}$$

常用激活函数： Sigmoid 、 Tanh 、 ReLU

ReLU: max(0, x)

默认选择

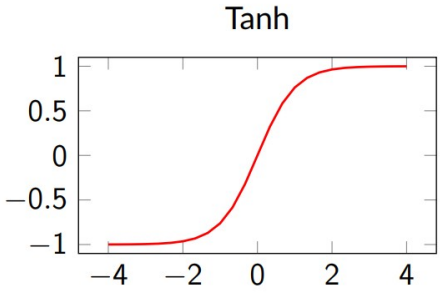


ReLU 让梯度更容易在深层网络中传播，使训练更快、更稳定。

优点： x > 0 时无梯度消失（斜率为 1）

缺点： ReLU 死亡（x < 0 时梯度恒为 0）。

Tanh: tanh(x)

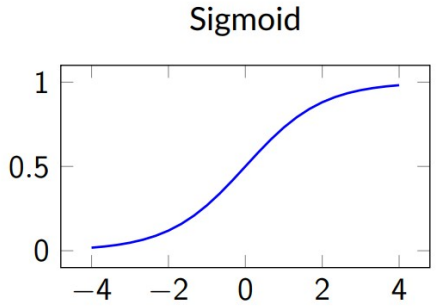


优点：以零为中心

缺点：梯度消失（x 很大或很小时梯度 ≈ 0）

反向传播时梯度相乘： 0.1 × 0.1 × 0.1 = 0.001 ，用于更新前面层的信号“消失”了。

Sigmoid: $\frac{1}{1+e^{-x}}$

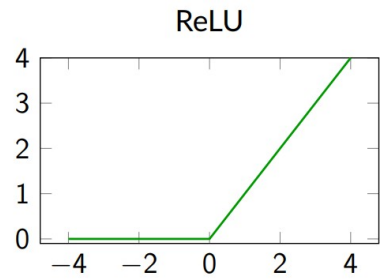


缺点：梯度消失（x 很大或很小时梯度 ≈ 0）

Common Activations: Sigmoid, Tanh, ReLU

ReLU: max(0, x)

the default

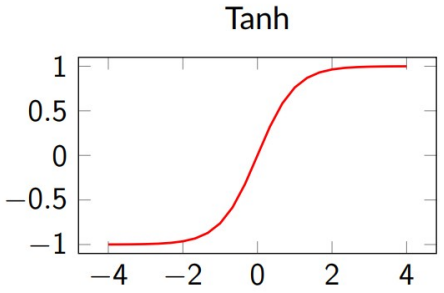


ReLU helps gradients flow more easily through deep networks, making training faster and more stable.

Good: No vanishing gradient for x > 0 (Slope is 1)

Bad: Dying ReLU (gradient is 0 for all x < 0).

Tanh: tanh(x)

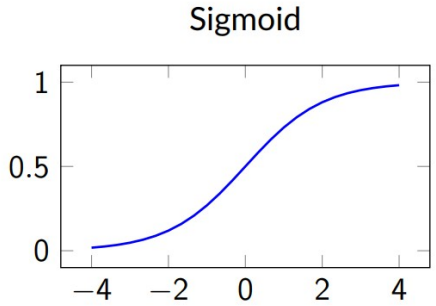


Good: Zero-Centered

Bad: Vanishing gradients (gradient ≈ 0 for large/small x)

During backpropagation, gradients are multiplied. 0.1 × 0.1 × 0.1 = 0.001. The signal to update the early layers "vanishes".

Sigmoid: $\frac{1}{1+e^{-x}}$



Bad: Vanishing gradients (gradient ≈ 0 for large/small x)

一种特殊的激活函数： Softmax

把得分向量（ logits ）转换为概率分布。

- 每个输出在 0 到 1 之间，所有输出之和为 1 。
- 通常不用作隐藏层的激活函数。
- 常用于多类分类的输出。

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

A Special Activation: Softmax

Converts a vector of scores (logits) into a probability distribution.

- Each output is between 0 and 1, and all outputs sum to 1.
- Not usually used as a hidden-layer activation.
- Commonly used for multi-class classification outputs.

$$\text{Softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

PyTorch 中的 MLP

nn.Linear in nn.Module

```
import torch
import torch.nn as nn
import torch.nn.functional as F
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        # Define the layers (weights are created automatically)
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, output_dim)
    def forward(self, x):
        # Define the computation flow
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.relu(x)
        x = self.fc3(x)
        return x
model = MLP(input_dim=784, hidden_dim=128, output_dim=10)
print(model)
```

MLP in PyTorch

nn.Linear in nn.Module

```
import torch
import torch.nn as nn
import torch.nn.functional as F
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        # Define the layers (weights are created automatically)
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, output_dim)
    def forward(self, x):
        # Define the computation flow
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.relu(x)
        x = self.fc3(x)
        return x
model = MLP(input_dim=784, hidden_dim=128, output_dim=10)
print(model)
```



卷积神经网络（CNN）



Convolutional Neural Networks (CNN)

卷积神经网络（CNN）

CNN 的提出是为了解决 MLP 的一个局限。

问题：能否直接对图像使用一个大型 MLP？

以一张“小”的 256x256 RGB 图像为例：

- 输入维度： $256 \times 256 \times 3 = 196,608$ 个特征。
- 若第一个隐藏层有 1,000 个神经元（并不多）……
- 权重数 $= 196,608 \times 1,000 =$ **1.966 亿**
- 这仅仅是一层。模型会庞大而缓慢，且很容易过拟合。

Convolutional Neural Networks (CNN)

CNN is proposed to address a limitation of MLP.

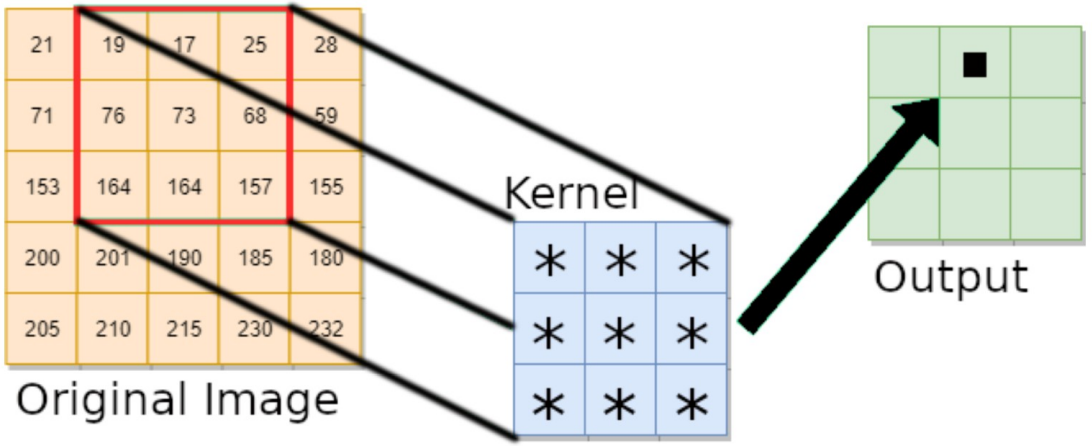
Problem: Can we just use a big MLP on an image?

Let’s try on a "small" 256x256 RGB image:

- Input dimensions: 256 × 256 × 3 = 196, 608 features.
- If our first hidden layer has 1,000 neurons (modest)…
- Number of weights = 196, 608 × 1, 000 = **196.6 Million**
- This is for one layer. The model would be enormous and slow, and could easily overfit.

卷积神经网络（CNN）

卷积运算：小滤波器（卷积核）在图像上滑动，充当局部特征检测器。



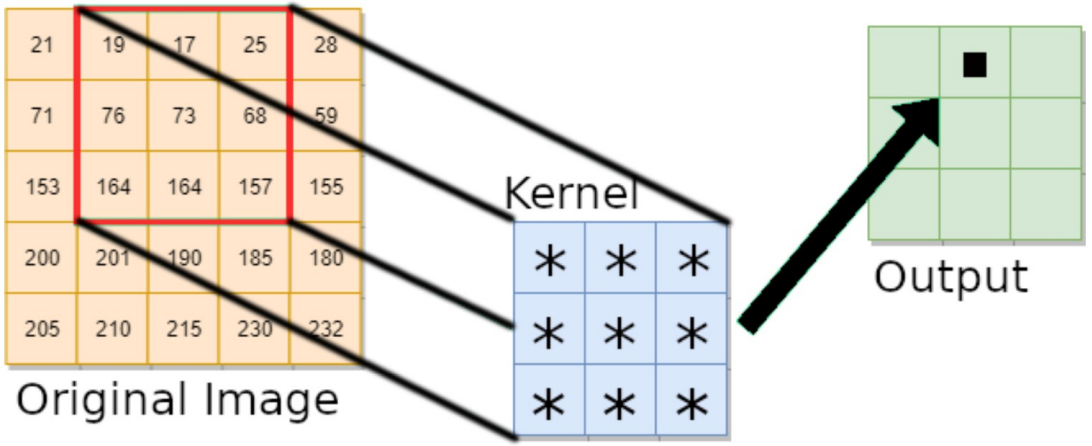
“滤波器”即卷积核（如 3x3 ）在输入图像上滑动。
每一步做逐元素相乘再求和（即点积）

思想：模式是局部的，相邻像素往往相关（如一只“眼睛”）。

与 MLP 相比：MLP 认为左上角像素与右下角像素的关联程度相同。

Convolutional Neural Networks (CNN)

Convolution Operation: A small filter (kernel) slides over the image as local feature detectors.



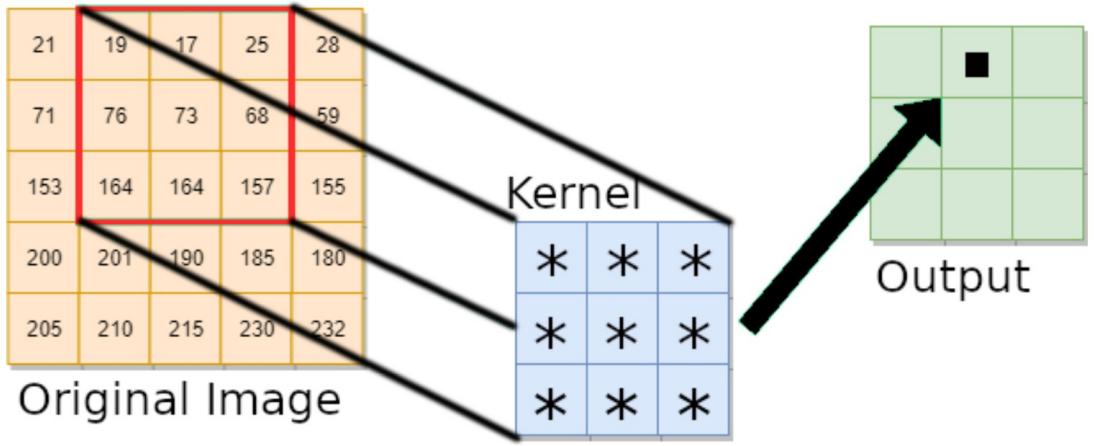
A "filter" or kernel (e.g., 3x3) slides over the input image. At each step, it computes an element-wise multiplication and sums the result (a dot product)

Idea: Patterns are local. Pixels near each other are often related (e.g., an "eye").

Compared with MLP: An MLP treats the top-left pixel and bottom-right as equally related.

卷积神经网络（CNN）

参数共享：各处使用相同的滤波器权重。



“滤波器”即卷积核（如 3x3）在输入图像上滑动。每一步做逐元素相乘再求和（即点积）

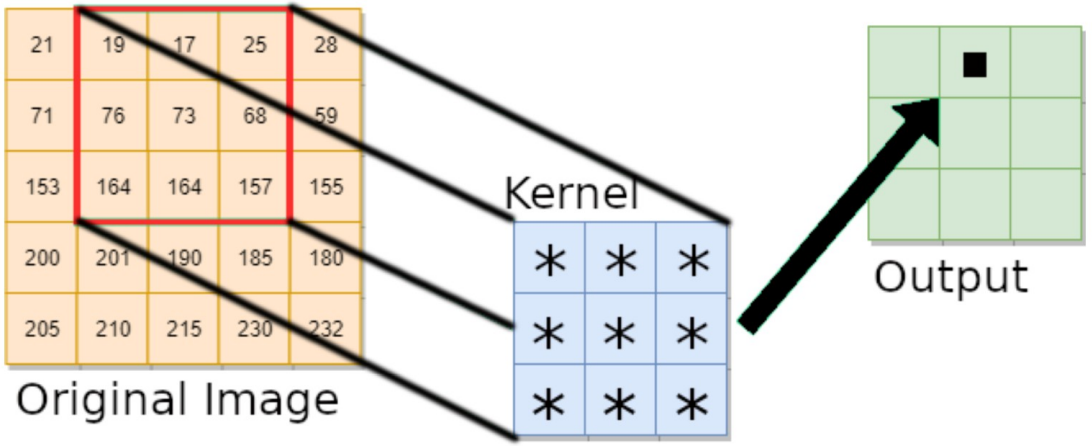
思想：特征具有平移不变性。
左上角的“猫”与右下角的“猫”是同一种“猫”。

与 MLP 相比：

- MLP 必须在每个可能的位置重新学习“猫”。
- 3x3 滤波器只需 9 个权重，而 MLP 需要 1.96 亿个。

Convolutional Neural Networks (CNN)

Parameter Sharing: Same filter weights used everywhere.



A "filter" or kernel (e.g., 3x3) slides over the input image. At each step, it computes an element-wise multiplication and sums the result (a dot product)

Idea: Features are translation-invariant. A "cat" in the top-left corner is the same "cat" as one in the bottom-right.

Compared with MLP:

- An MLP would have to re-learn "cat" at every possible position.
- 3x3 filter needs only 9 weights vs. 196M for MLP.

CNN 中的通道

实际的 CNN 通常有多个通道。

- RGB 图像有 3 个**输入通道**：R、G、B。
- 一个滤波器使用全部输入通道，生成**一张特征图（一个输出通道）**。
- 多个滤波器产生多个输出通道。

示例：64 个滤波器 → 64 个输出通道。

Channels in CNN

A real CNN usually has multiple channels.

- An RGB image has 3 **input channels**: R, G, and B.
- One filter uses all input channels and produces **one feature map (one output channel)**.
- Multiple filters produce multiple output channels.

Example: 64 filters → 64 output channels.

CNN 中的通道

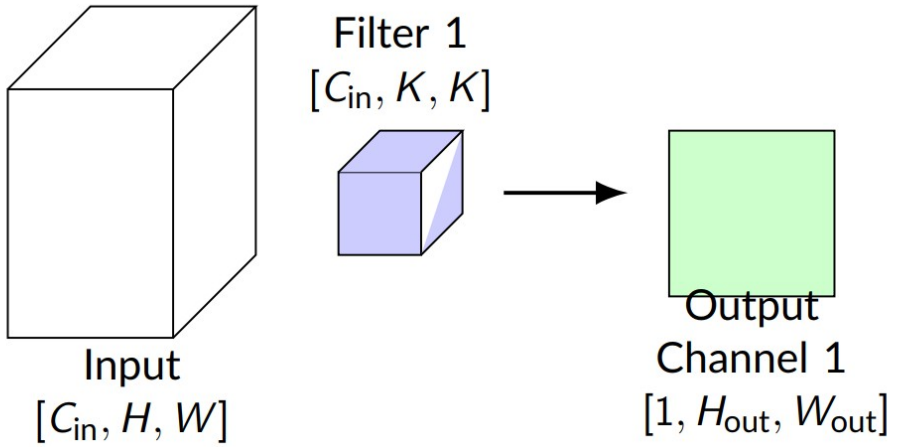
- 有 C_{in} 个通道（如 RGB 为 3）的输入不是二维的，而是三维的。
- 因此滤波器也必须是三维的，且必须与输入深度相同。
- RGB 图像上的“3x3 滤波器”实际上是 3x3x3 的权重块。
- 这个三维滤波器在输入上滑动，同时覆盖全部三个输入通道，计算一次点积（$3 \times 3 \times 3 = 27$ 次乘法），得到一个数。
- 要得到 C_{out} （如 64）个输出通道，需要 64 个不同的 3x3x3 滤波器。

```
# 3 input channels (RGB)
# 64 output channels (64 filters)
# 3x3 kernel size
conv1 = nn.Conv2d(in_channels=3, out_channels=64,
    kernel_size=3)
```

Input Shape: $[\text{Batch}, C_{in}, H_{in}, W_{in}]$

Filter Shape: $[C_{out}, C_{in}, K_H, K_W]$

Output Shape: $[\text{Batch}, C_{out}, H_{out}, W_{out}]$



Channels in CNN

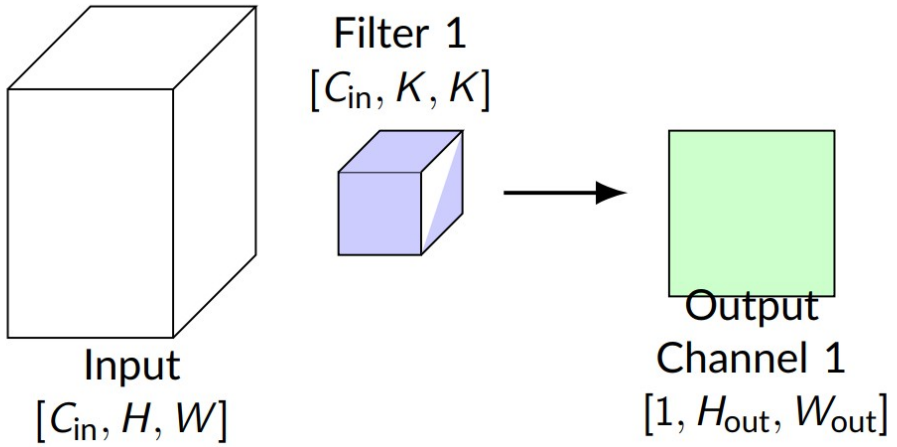
- An input with C_{in} channels (e.g., 3 for RGB) is not 2D, it's 3D.
- Therefore, the filter must also be 3D. It must have the same depth as the input.
- A "3x3 filter" on an RGB image is actually a 3x3x3 block of weights.
- This 3D filter slides over the input while covering all three input channels, computes one dot product (with 3 × 3 × 3 = 27 multiplications), and produces one number.
- To get C_{out} (e.g., 64) output channels, we need 64 different 3x3x3 filters.

```
# 3 input channels (RGB)
# 64 output channels (64 filters)
# 3x3 kernel size
conv1 = nn.Conv2d(in_channels=3, out_channels=64,
    kernel_size=3)
```

Input Shape: $[\text{Batch}, C_{in}, H_{in}, W_{in}]$

Filter Shape: $[C_{out}, C_{in}, K_H, K_W]$

Output Shape: $[\text{Batch}, C_{out}, H_{out}, W_{out}]$



CNN 中的步幅与填充

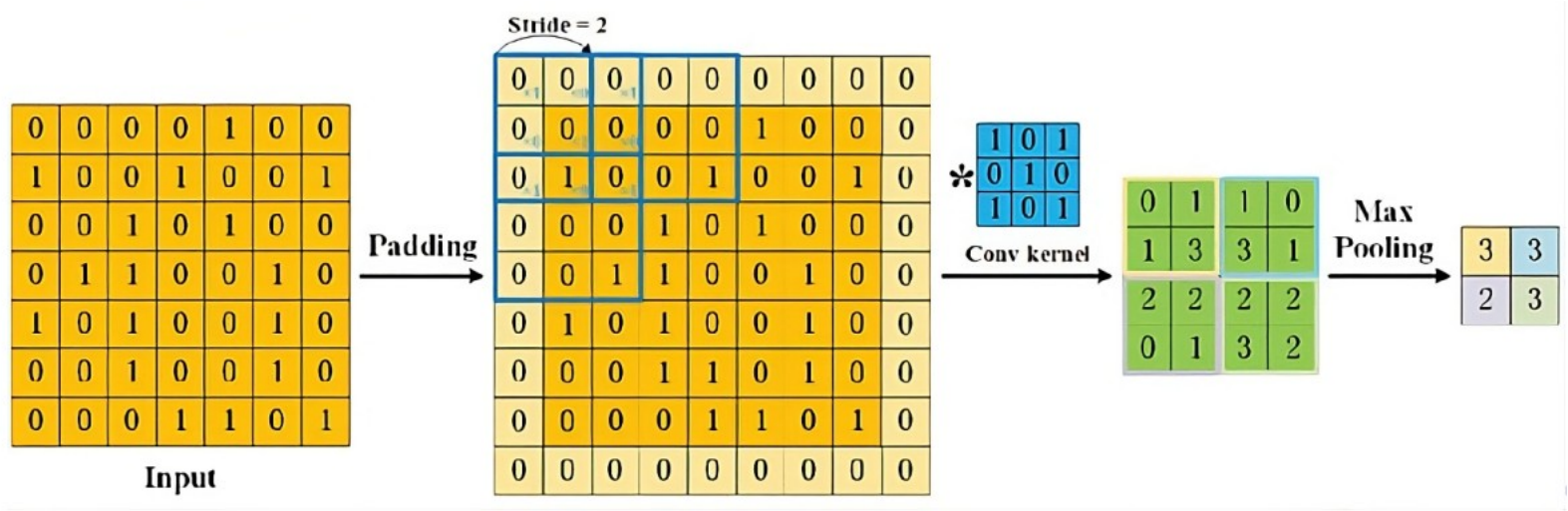
卷积有两个控制参数

步幅

- **滤波器每次“移动”多少个像素。**
- 默认 stride=1 （每次移动 1 个像素）。
- stride=2 会隔一个像素移动，对图像下采样（输出尺寸减半）。

填充

- 卷积会“缩小”图像（5x5 输入配 3x3 滤波器得到 3x3 输出）。
- **在输入边界周围补零以保持尺寸。**
- 经验法则：对 K × K 滤波器，用 padding = (K-1)/2 保持尺寸不变。



Stride and Padding in CNN

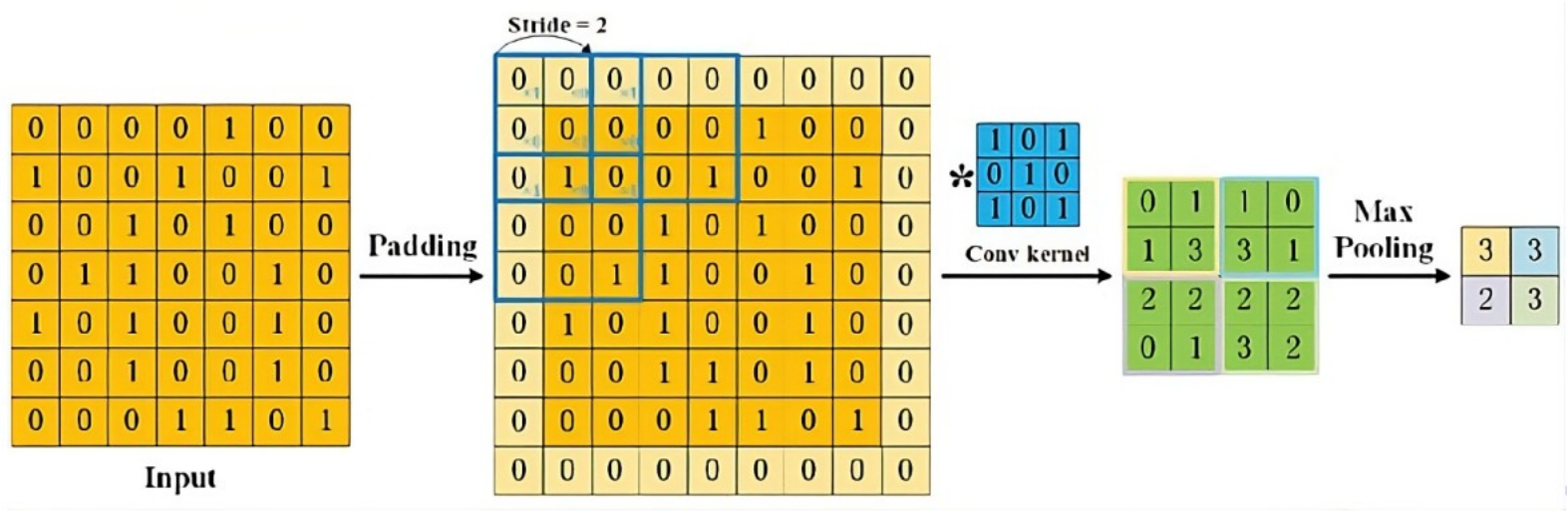
We have two controls for the convolution

Stride

- **How many pixels to "step" the filter at a time.**
- Default stride=1 (moves 1 pixel at a time).
- stride=2 will skip every other pixel, downsampling the image (output will be half the size).

Padding

- Convolutions “shrink” the image (a 5x5 input with 3x3 filter gives a 3x3 output).
- **We pad the input with zeros around the border to preserve its size.**
- Rule of thumb: For a K × K filter, use padding = (K-1)/2 to keep the size the same.



PyTorch 中的 Conv2D： nn.Conv2d

```
import torch
import torch.nn as nn
# Input tensor: [batch, channels, height, width]
x = torch.randn(1, 3, 64, 64)  # 1 image, 3 channels, 64
    x64 pixels
# Conv2d with stride and padding
conv2d = nn.Conv2d(
    in_channels=3,       # RGB input
    out_channels=32,     # 32 filters
    kernel_size=3,       # 3x3 filters
    stride=2,            # Stride: move 2 pixels at a
        time
    padding=1            # Padding: pad height and width
        dimensions
)
output = conv2d(x)
print(f"Input shape:  {x.shape}")       # [1, 3, 64, 64]
print(f"Output shape: {output.shape}") # [1, 32, 32, 32]
```

Conv2D in PyTorch: nn.Conv2d

```
import torch
import torch.nn as nn
# Input tensor: [batch, channels, height, width]
x = torch.randn(1, 3, 64, 64)  # 1 image, 3 channels, 64
    x64 pixels
# Conv2d with stride and padding
conv2d = nn.Conv2d(
    in_channels=3,       # RGB input
    out_channels=32,     # 32 filters
    kernel_size=3,       # 3x3 filters
    stride=2,            # Stride: move 2 pixels at a
        time
    padding=1            # Padding: pad height and width
        dimensions
)
output = conv2d(x)
print(f"Input shape:  {x.shape}")       # [1, 3, 64, 64]
print(f"Output shape: {output.shape}") # [1, 32, 32, 32]
```

计算输出尺寸

这是设计 CNN 的关键公式。

已知： W_{in} = 输入宽度， K = 卷积核（滤波器）大小， P = 填充， S = 步幅

输出宽度 W_{out} 为：

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1$$

同一公式也适用于高度 H 。

示例：输入为 32 × 32 ，滤波器为 3 × 3 （ K = 3 ），步幅为 1 （ S = 1 ）。

- 无填充（ P = 0 ）： $W_{out} = (32 - 3 + 0)/1 + 1 = 30$ ，输出为 30 × 30 （**变小了**）。
- “ Same ”填充（ P = 1 ）： $W_{out} = (32 - 3 + 2)/1 + 1 = 32$ ，输出为 32 × 32 （保持不变）

Calculating Output Size

This is a critical formula for designing CNNs.

Given: W_{in} = Input Width, K = Kernel (Filter) Size, P = Padding, S = Stride

The output width W_{out} is:

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K + 2P}{S} \right\rfloor + 1$$

The same formula applies to the height H.

Example: Input is 32 × 32. Filter is 3 × 3 (K = 3). Stride is 1 (S = 1).

- No Padding (P = 0): W_{out} = (32 − 3 + 0)/1 + 1 = 30. Output is 30 × 30 (**it shrinks**).
- With "Same" Padding (P = 1): W_{out} = (32 − 3 + 2)/1 + 1 = 32. Output is 32 × 32 (it stays the same)

池化层

思想：对特征图下采样。

- 最大池化（nn.MaxPool2d）
  - 取一个局部窗口（如 2 × 2），保留其中的**最大**值。
  - 效果：保留最强的特征信号。

```
import torch.nn as nn
# Max Pooling
max_pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
```

- 平均池化（nn.AvgPool2d）
  - 取一个局部窗口，改为输出其**平均**值。
  - 效果：给出局部区域更平滑的概括。

```
import torch.nn as nn
# Average Pooling
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2, padding=0)
```

池化没有可学习参数。

Pooling Layers

Idea: downsample the feature map.

- Max Pooling (nn.MaxPool2d)
  - Takes a local window (e.g., 2 × 2) and keeps the **maximum** value.
  - Effect: Keeps the strongest feature signal.

```
import torch.nn as nn
# Max Pooling
max_pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)
```

- Average Pooling (nn.AvgPool2d)
  - Takes a local window and outputs the **average** value instead.
  - Effect: Gives a smoother summary of the local region.

```
import torch.nn as nn
# Average Pooling
avg_pool = nn.AvgPool2d(kernel_size=2, stride=2, padding=0)
```

Pooling has no learnable parameters.

为什么要池化?

计算效率：

- 步幅为 2 的 2×2 池化层把 32×32 的特征图缩小为 16×16 。
- 特征图缩小为原来的四分之一，后续层所需计算更少。

局部平移不变性：

- 最大池化使网络对特征位置的微小变化不那么敏感。
- 位于 $(4, 4)$ 与 $(4, 5)$ 的边缘可能得到相同的最大输出。
- 这有助于模型更好地泛化

Why Pooling?

Computational Efficiency:

- A 2 × 2 pooling layer with stride 2 reduces a 32 × 32 feature map to 16 × 16.
- This reduces the feature map to one-quarter of its original size, so later layers require less computation.

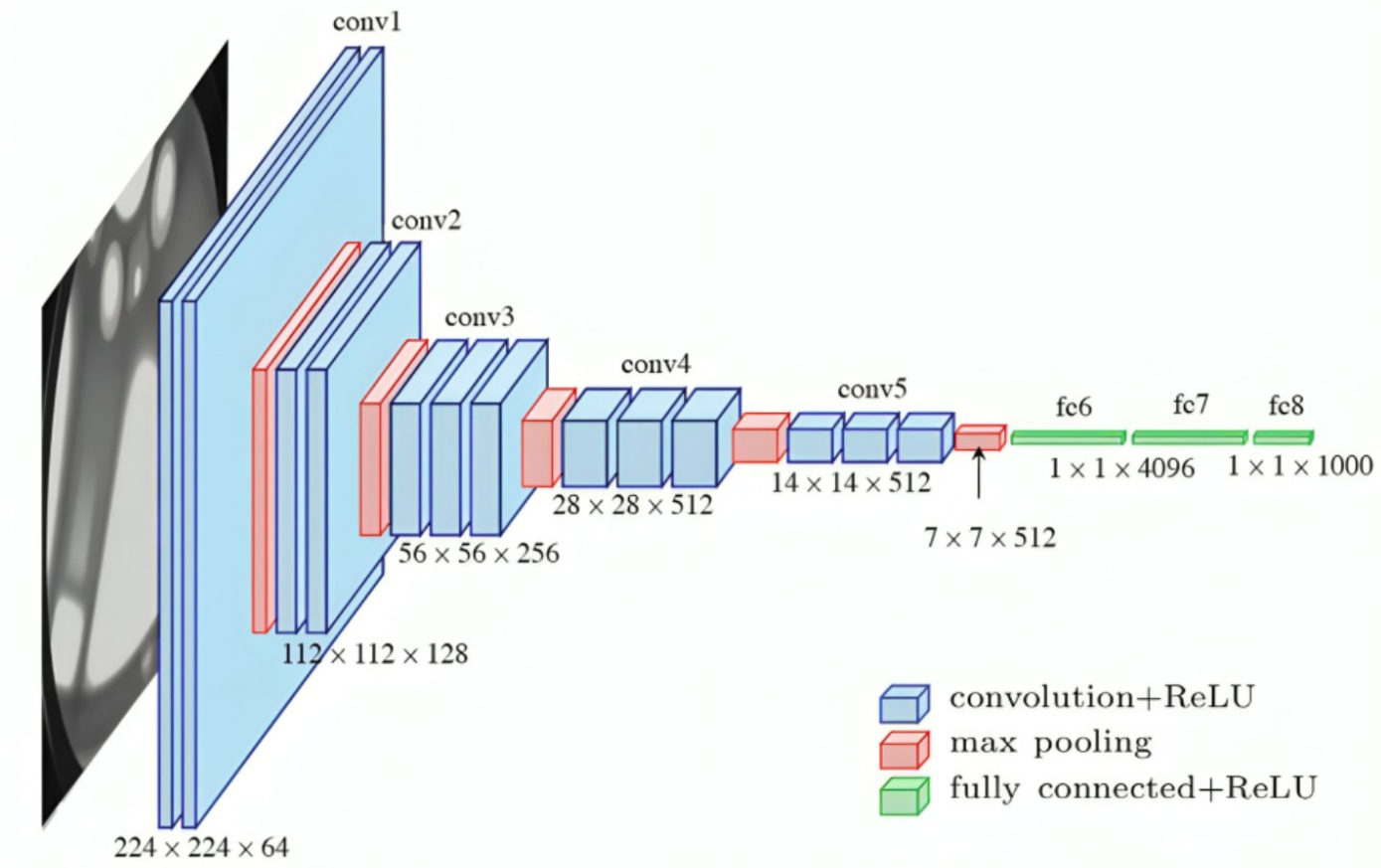
Local Translation Invariance:

- Max pooling makes the network less sensitive to small changes in the position of a feature.
- An edge in position (4, 4) and an edge in (4, 5) might both result in the same max output.
- This helps the model generalize better

完整的 CNN 架构（如 VGG 风格）

典型的 CNN 架构是一种重复模式：

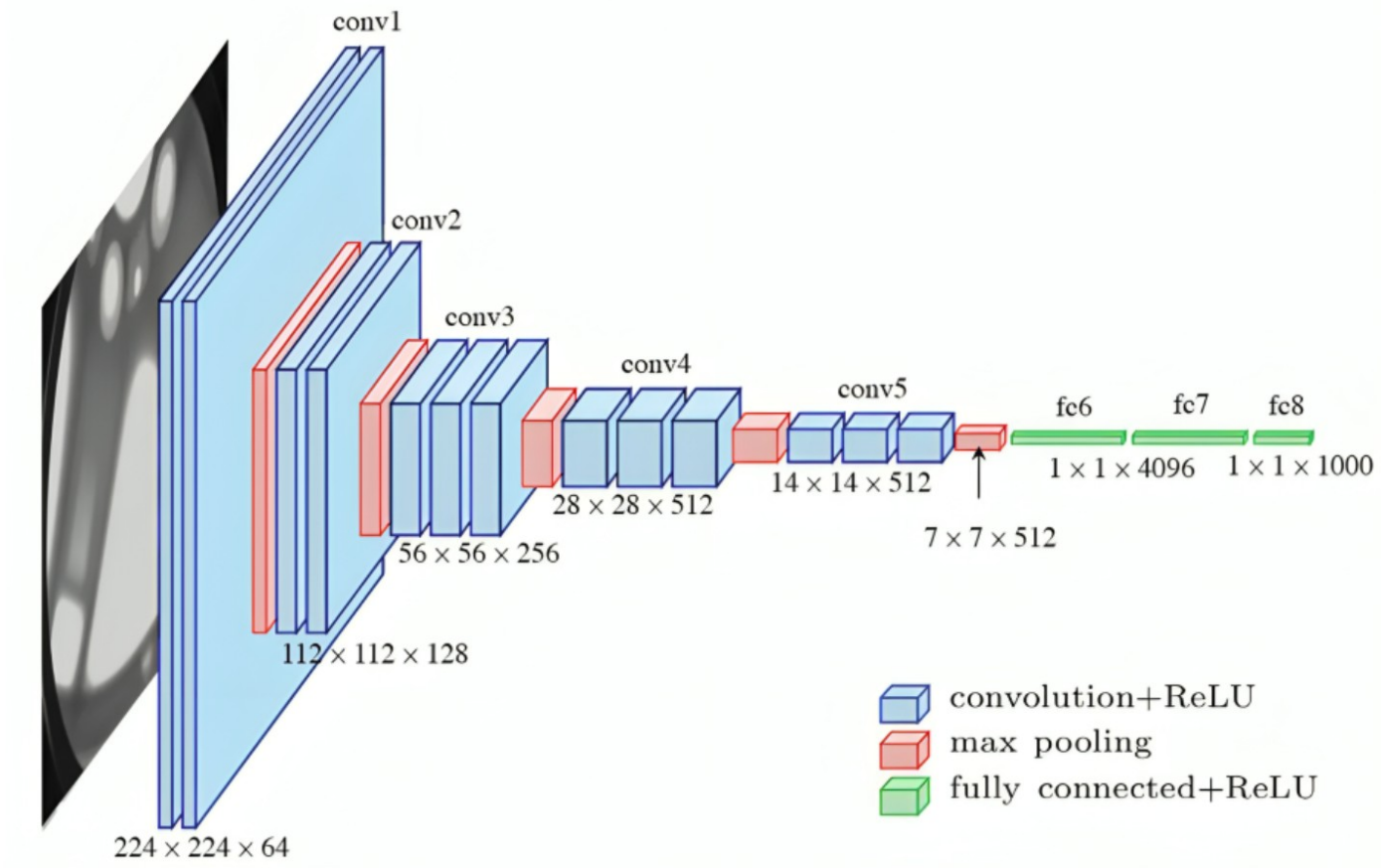
(CONV → ReLU → POOL) × N → (FC → ReLU) × M → FC (Output) ：



A Full CNN Architecture (e.g., VGG-style)

A typical CNN architecture is a repeating pattern:

(CONV → ReLU → POOL) × N → (FC → ReLU) × M → FC (Output) :



PyTorch 中的简单 CNN

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Convolutional Base
        self.conv_base = nn.Sequential(
            # Input: 3 x 32 x 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2), # -> 32 x 16 x 16
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)  # -> 64 x 8 x 8
        )
        # Classifier Head
        self.classifier = nn.Sequential(
            nn.Flatten(),  # -> 64 * 8 * 8 = 4096
            nn.Linear(64 * 8 * 8, 512),
            nn.ReLU(),
            nn.Linear(512, num_classes)
        )
    def forward(self, x):
        x = self.conv_base(x)
        x = self.classifier(x)
        return x
```

A Simple CNN in PyTorch

```
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # Convolutional Base
        self.conv_base = nn.Sequential(
            # Input: 3 x 32 x 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2), # -> 32 x 16 x 16
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)  # -> 64 x 8 x 8
        )
        # Classifier Head
        self.classifier = nn.Sequential(
            nn.Flatten(),  # -> 64 * 8 * 8 = 4096
            nn.Linear(64 * 8 * 8, 512),
            nn.ReLU(),
            nn.Linear(512, num_classes)
        )
    def forward(self, x):
        x = self.conv_base(x)
        x = self.classifier(x)
        return x
```

残差连接的动机

我们知道堆叠层能构建更好的特征。为何不堆 100 层？甚至 1000 层？

梯度消失
- 层数越多，前面层的梯度会“消失”，它们便停止学习。
- ReLU 有帮助，但不能完全解决

退化
- 简单堆叠 100 层的效果可能比 20 层的版本还差。
- 很深的网络甚至难以学会简单的恒等映射（即直接传递输入 $H(x) = x$）

Motivation for Residual Connections

We know stacking layers builds better features. Why not stack 100 layers? Or 1000?

Vanishing Gradients

- With more layers, the gradients for early layers "vanish", and they stop learning.
- ReLU helps, but doesn’t fix it entirely

Degradation

- Simply stacking 100 layers can perform worse than a 20-layer version.
- A very deep network may even have trouble learning a simple identity mapping (i.e., just passing the input through H(x) = x)

残差连接（ResNet）

- **跳跃连接**把原始输入 x 直接加到模块输出上。
- 模块不学习完整变换 H(x)，而是**学习残差 F(x)： H(x)=F(x)+x** 。
- 若期望映射为 H(x)=x，模块只需学会让 F(x) 接近零。
- 跳跃连接还有助于梯度在网络中反向传播。

Plain Block

$x \rightarrow$ Conv Block $\rightarrow H(x)$

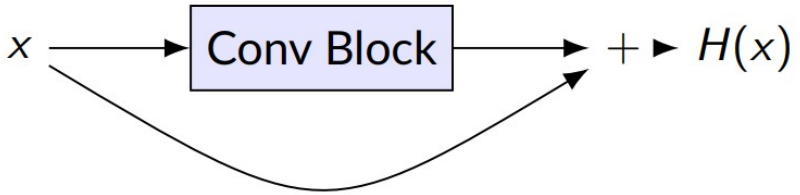
The block must learn the entire transformation $H(x)$.



Residual Block

$H(x) = F(x) + x$

The block only learns the *residual* $F(x)$.



Residual Connections (ResNet)

- A **skip connection** adds the original input x directly to the block output.
- Instead of learning the full transformation H(x), the block **learns the residual F(x): H(x)=F(x)+x.**
- If the desired mapping is H(x)=x, the block can simply learn to make F(x) close to zero.
- The skip connection also helps gradients flow backward through the network.

Plain Block

$x \rightarrow$ Conv Block $\rightarrow H(x)$

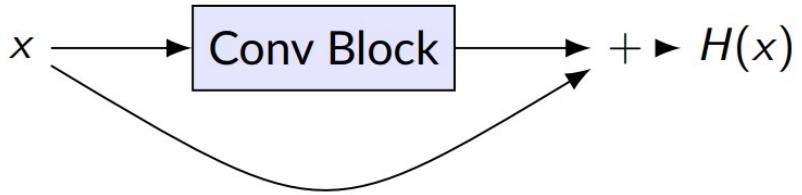
The block must learn the entire transformation $H(x)$.



Residual Block

$H(x) = F(x) + x$

The block only learns the *residual* $F(x)$.



PyTorch 中的 ResNet 模块

```
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3,
            padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3,
            padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

    def forward(self, x):
        # Store the original input for the skip connection
        identity = x
        # Pass through the block
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        # Add the skip connection
        out = out + identity
        # Apply final activation
        out = F.relu(out)
        return out
```

ResNet Block in PyTorch

```python
class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3,
            padding=1)
        self.bn1 = nn.BatchNorm2d(channels)
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3,
            padding=1)
        self.bn2 = nn.BatchNorm2d(channels)

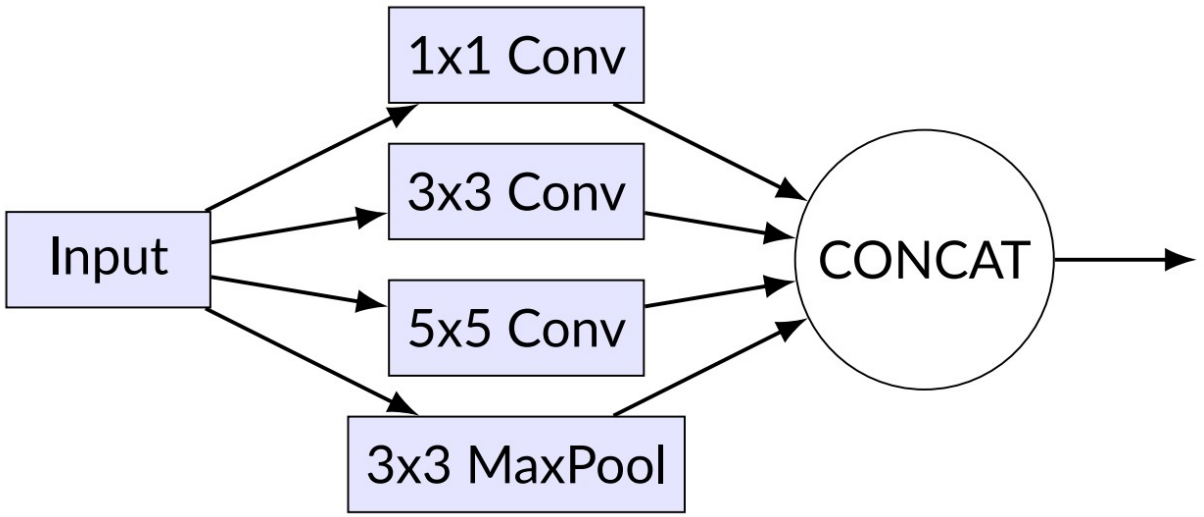
    def forward(self, x):
        # Store the original input for the skip connection
        identity = x
        # Pass through the block
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        # Add the skip connection
        out = out + identity
        # Apply final activation
        out = F.relu(out)
        return out
```

常用模块： Inception 模块

问题：哪种卷积核大小最好？ 3x3？ 5x5？ 1x1？

思想（GoogLeNet / Inception）：为何不全都用?

- “Inception 模块”并行运行多个卷积分支，并拼接它们的输出。
- 这使网络可以变得“更宽”，而不仅仅是“更深”。

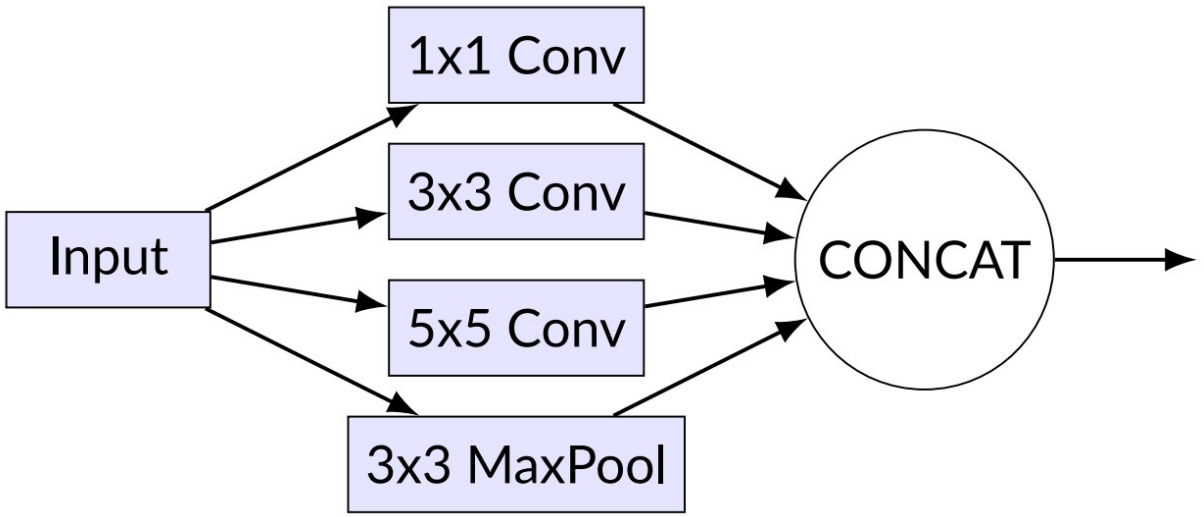


Common Blocks: Inception Module

Problem: What kernel size is best? 3x3? 5x5? 1x1?

Idea (GoogLeNet / Inception): Why not all of them?

- An "Inception module" runs multiple convolutional branches in parallel and concatenates their outputs.
- This allows the network to be "wider" instead of just "deeper".





序列建模与 RNN



Sequence Modeling and RNN

CNN/MLP 处理序列的问题

什么是序列数据?

- **顺序很重要**的数据。
- 文本：“ man bites dog ”（人咬狗） vs. “ dog bites man ”（狗咬人）
- 时间序列：股价、天气数据
- 音频：波形

为什么标准 MLP/CNN 不适合序列建模?

- **没有明确的“时间”或顺序概念：**它们不会在步与步之间追踪时间或顺序。
- **固定尺寸输入：**标准 MLP 和许多 CNN 分类器通常要求固定尺寸的输入。
- **没有“记忆”：**没有把早期步骤的信息带到后续步骤的显式记忆。

Problem with CNNs/MLPs for Sequences

What is Sequence Data?

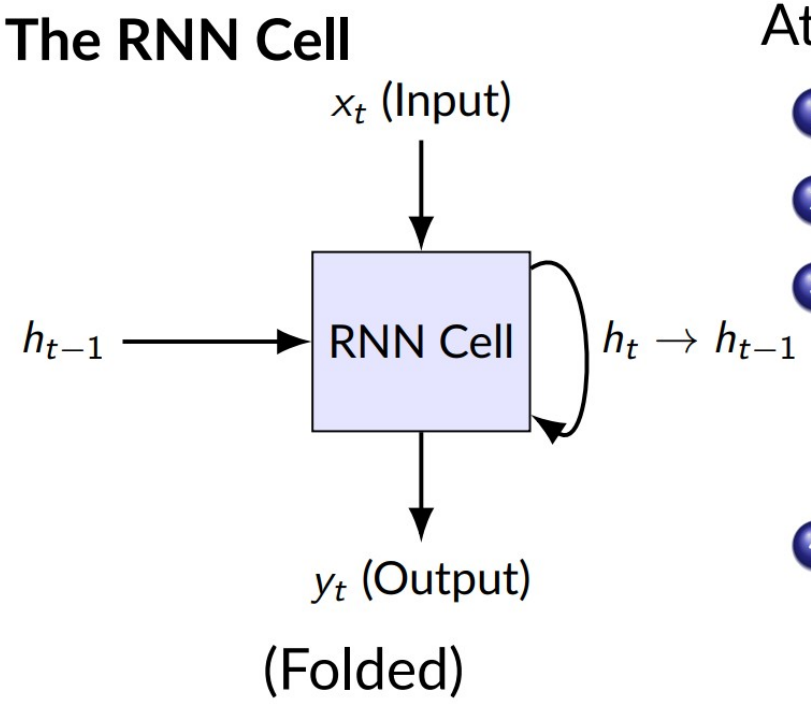
- Data where **order matters**.
- Text: "man bites dog" vs. "dog bites man"
- Time Series: Stock prices, weather data
- Audio: Waveforms

Why are standard MLPs/CNNs not ideal for sequence modeling?

- **No explicit sense of "time" or order:** They do not keep track of time or order from one step to the next.
- **Fixed-size input:** Standard MLPs and many CNN classifiers usually expect a fixed-size input.
- **No "memory":** There is no explicit memory that carries information from earlier steps to later steps.

解决方案：循环神经网络（RNN）

思想：带有“**循环**”的网络，维护一个“**记忆**”或“状态”
隐藏状态 h_t 是网络的“记忆”，概括了网络到时刻 t 为止看到的内容。



At each time step t, the RNN cell:

1. Takes the **current input** x_t.
2. Takes the **previous hidden state** h_{t-1}.
3. Computes the **new hidden state** h_t:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

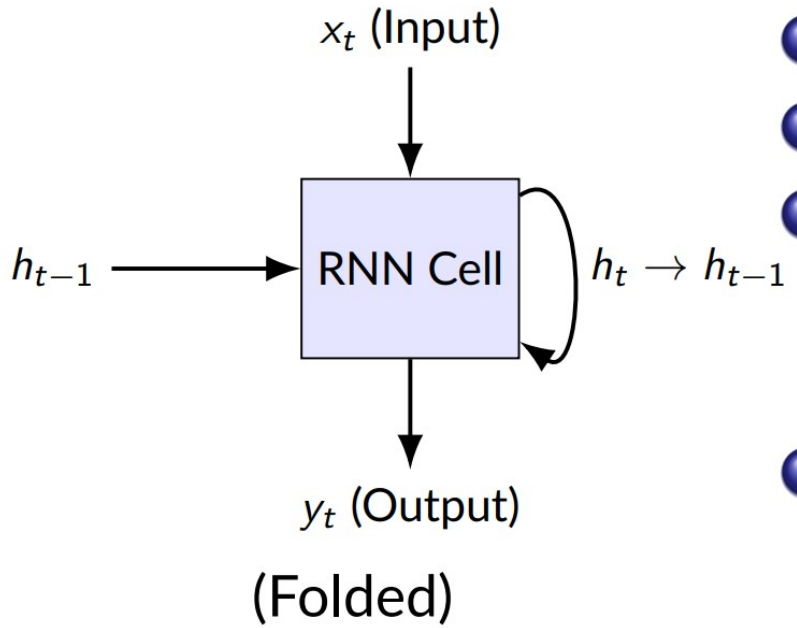
4. (Optionally) computes an output y_t:

$$y_t = W_{hy}h_t + b_y$$

Solution: Recurrent Neural Networks (RNNs)

Idea: A network with a "**loop**" that maintains a "**memory**" or "state"
The hidden state h_t is the network's "memory". It's a summary of what the network has seen up to time t.

The RNN Cell



(Folded)

At each time step t, the RNN cell:

1. Takes the **current input** x_t.
2. Takes the **previous hidden state** h_{t-1}.
3. Computes the **new hidden state** h_t:

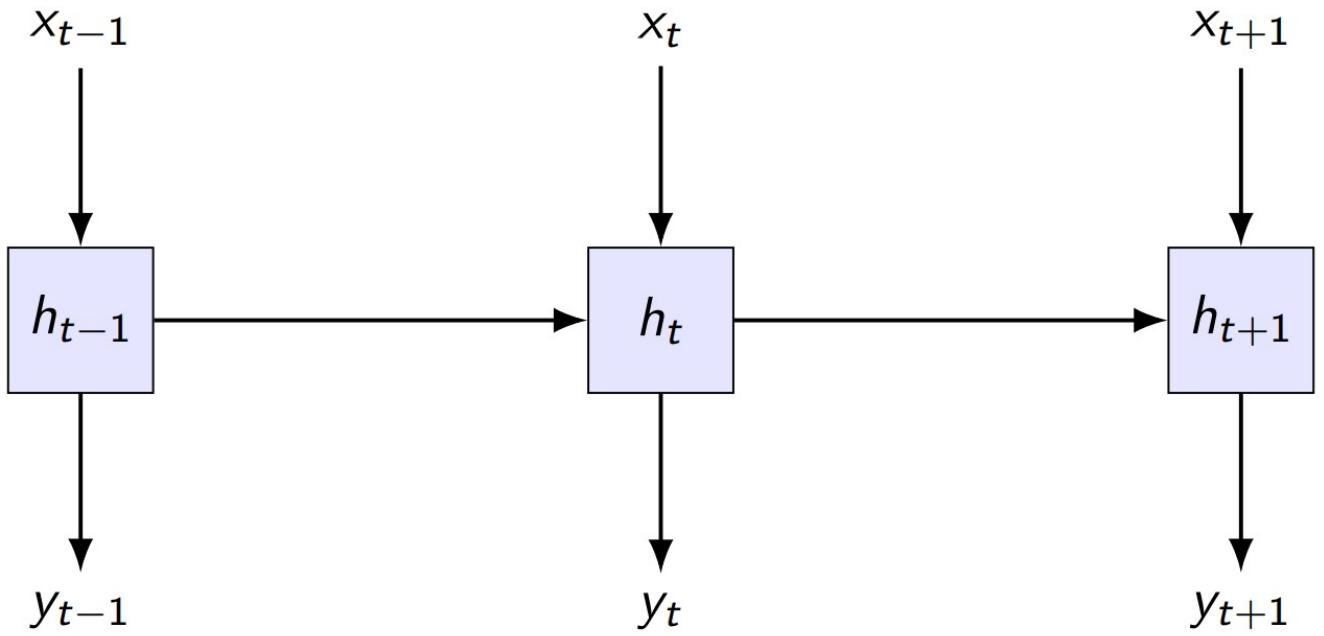
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

4. (Optionally) computes an output y_t:

$$y_t = W_{hy}h_t + b_y$$

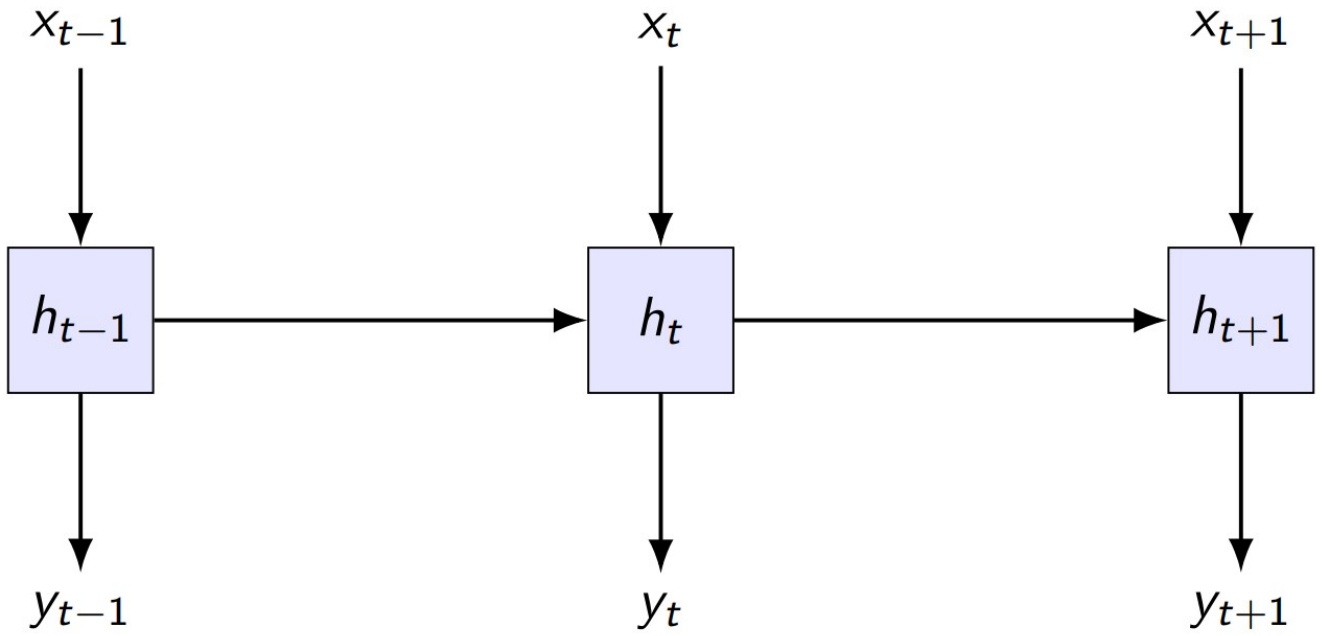
展开 RNN

- RNN 的循环可以**沿时间“展开”**为一串重复的步骤。
- 展开后，它就像一个**每个时间点对应一步的深层网络**。
- 所有时间步**共享相同的权重**。
- 训练使用**随时间反向传播（BPTT）**，即在展开的计算图上做标准反向传播。



Unrolling the RNN

- The RNN loop can be **“unrolled” through time** into a sequence of repeated steps.
- After unrolling, it looks like a **deep network with one step at each time point**.
- All time steps **share the same weights**.
- Training is done with **Backpropagation Through Time (BPTT)**, which is just standard backpropagation on the unrolled computation graph.



PyTorch 中的 RNN： nn.RNN

```
# Hyperparameters
input_size = 10   # e.g., dimension of word embedding
hidden_size = 20  # dimension of our "memory"
num_layers = 2    # number of stacked RNN layers
batch_size = 5
seq_len = 7       # length of our sentences
# Initialize the RNN layer
rnn = nn.RNN(input_size=input_size,
             hidden_size=hidden_size,
             num_layers=num_layers,
             batch_first=True) # Makes shape [Batch, Seq, Feat]
# Create dummy input data
# Shape: [batch_size, sequence_length, input_size]
dummy_input = torch.rand(batch_size, seq_len, input_size)
# Pass it through the RNN
# output: hidden state for *every* time step
# h_n: hidden state for the *last* time step
output, h_n = rnn(dummy_input)
print("Input shape:", dummy_input.shape)
print("Output shape:", output.shape)
print("Last hidden shape:", h_n.shape)
```

RNN in PyTorch: nn.RNN

```
# Hyperparameters
input_size = 10   # e.g., dimension of word embedding
hidden_size = 20  # dimension of our "memory"
num_layers = 2    # number of stacked RNN layers
batch_size = 5
seq_len = 7       # length of our sentences
# Initialize the RNN layer
rnn = nn.RNN(input_size=input_size,
             hidden_size=hidden_size,
             num_layers=num_layers,
             batch_first=True) # Makes shape [Batch, Seq, Feat]
# Create dummy input data
# Shape: [batch_size, sequence_length, input_size]
dummy_input = torch.rand(batch_size, seq_len, input_size)
# Pass it through the RNN
# output: hidden state for *every* time step
# h_n: hidden state for the *last* time step
output, h_n = rnn(dummy_input)
print("Input shape:", dummy_input.shape)
print("Output shape:", output.shape)
print("Last hidden shape:", h_n.shape)
```

门控 RNN

问题： RNN 的“记忆” h_t 不好用，它必须同时承担两项工作：

- 作为当前时间步的输出（用于预测）。
- 作为所有未来时间步的“记忆”。

解决方案：门控 RNN

- 建立一个与输出 (ht) 分开的、受保护的“**记忆**”（细胞状态 ct ）。
- 用“**门**”（乘性的 σ 函数）控制进出这一受保护记忆的信息流

由此产生：

- **LSTM** （长短期记忆）
- **GRU** （门控循环单元）

Gated RNNs

Problem: The RNN's "memory" h_t is bad. It has to be used for two jobs at once:

- Be the output for the current time step (for prediction).
- Be the "memory" for all future time steps.

Solution: Gated RNNs

- create a separate, protected "**memory**" (the Cell State, ct) that is separate from the output (ht).
- Use "**gates**" (multiplicative σ functions) to control the flow of information into and out of this protected memory

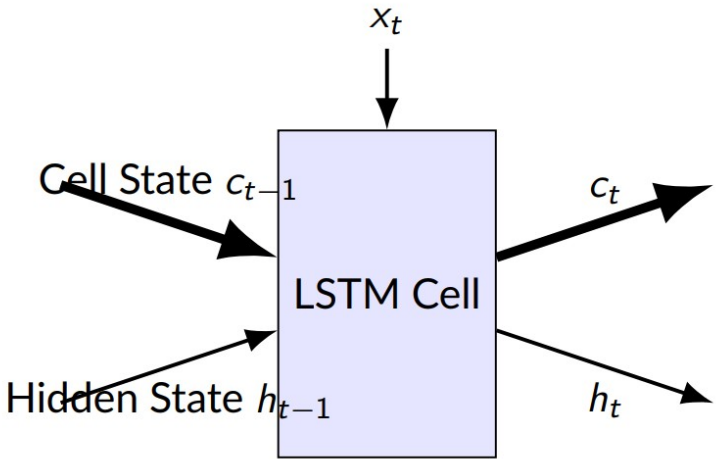
This leads to:

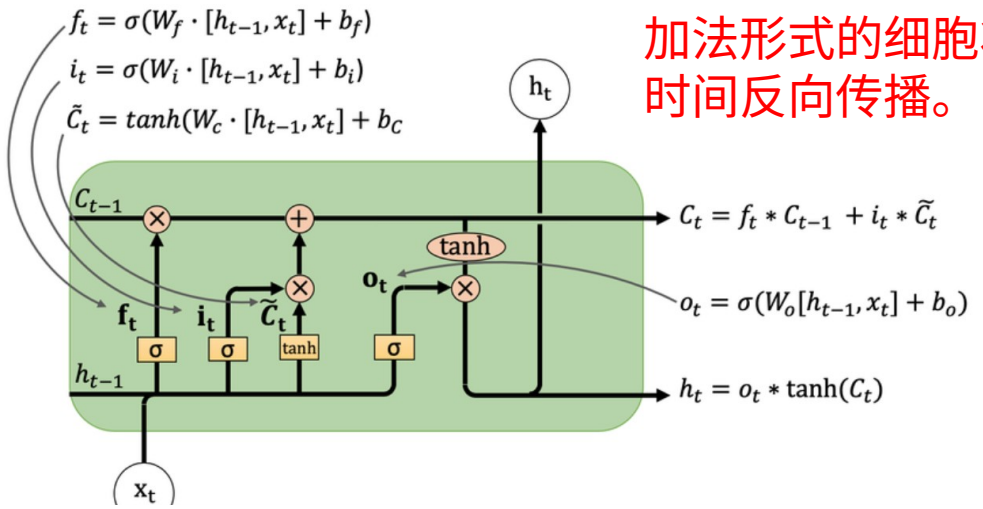
- **LSTM** (Long Short-Term Memory)
- **GRU** (Gated Recurrent Unit)

长短期记忆（LSTM）

LSTM 单元是更复杂的 RNN 单元，有两个状态：

- h_t（隐藏状态）“输出”状态，用于时刻 t 的预测。
- c_t（细胞状态）“记忆”状态，随时间传递信息。
- **网络用“门”对细胞状态进行读取、擦除和写入。**
- 遗忘门（f_t）：旧记忆 c_{t-1} 应遗忘多少？
- 输入门（i_t）：应加入多少新信息？
- 输出门（o_t）：“（更新后的）细胞状态 c_t 中哪部分应作为新的隐藏状态 h_t 输出？”



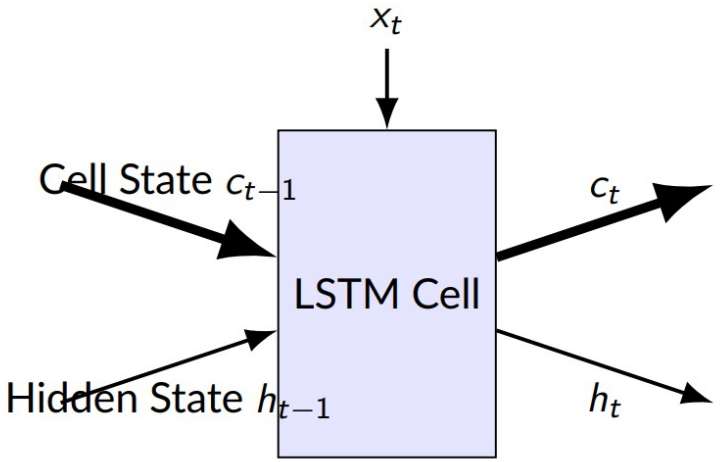


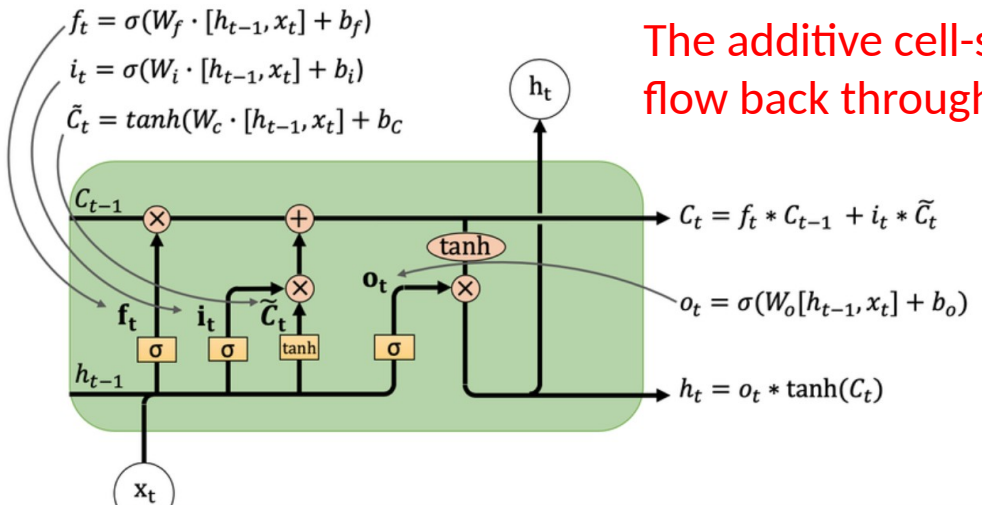
加法形式的细胞状态通路使梯度更容易沿时间反向传播。

Long Short-Term Memory (LSTM)

An LSTM cell is a more complex RNN cell with two states:

- h_t (Hidden State) The "output" state. Used for predictions at time t.
- c_t (Cell State) The "memory" state. Carry information through time.
- **The network uses "gates" to read from, erase from, and write to cell state.**
- Forget Gate (f_t): How much of the old memory c_{t-1} should we forget?
- Input Gate (i_t) : How much of the new information should we add?
- Output Gate (o_t) : "What part of the (updated) cell state c_t should we output as the new hidden





The additive cell-state path helps gradients flow back through time more easily.

PyTorch 中的 LSTM

```
# Initialize an LSTM layer
# API is identical to nn.RNN
lstm = nn.LSTM(input_size=input_size,
               hidden_size=hidden_size,
               num_layers=num_layers,
               batch_first=True) # Very important!
```

```
# LSTM returns (output, (h_n, c_n))
# h_n = final hidden state, c_n = final cell state
output, (h_n, c_n) = lstm(dummy_input)
```

LSTM in PyTorch

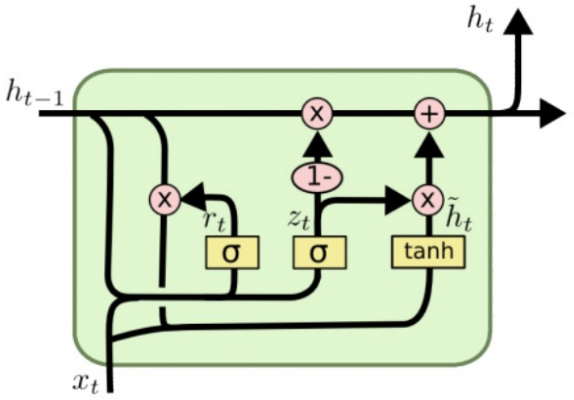
```
# Initialize an LSTM layer
# API is identical to nn.RNN
lstm = nn.LSTM(input_size=input_size,
               hidden_size=hidden_size,
               num_layers=num_layers,
               batch_first=True) # Very important!
```

```
# LSTM returns (output, (h_n, c_n))
# h_n = final hidden state, c_n = final cell state
output, (h_n, c_n) = lstm(dummy_input)
```

门控循环单元（GRU）

GRU 在许多任务上与 LSTM 效果相当，但训练更快（**参数更少**）。

- 它把隐藏状态和细胞状态合并为**一个隐藏状态 h_t**。
- 它只有**两个门**:
  - 更新门（z_t）：融合“遗忘门”与“输入门”，决定保留多少 h_{t-1}、加入多少新候选 $\tilde{h}_t$。
  - 重置门（r_t）：决定计算新候选 $\tilde{h}_t$ 时使用多少先前记忆 h_{t-1}。



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

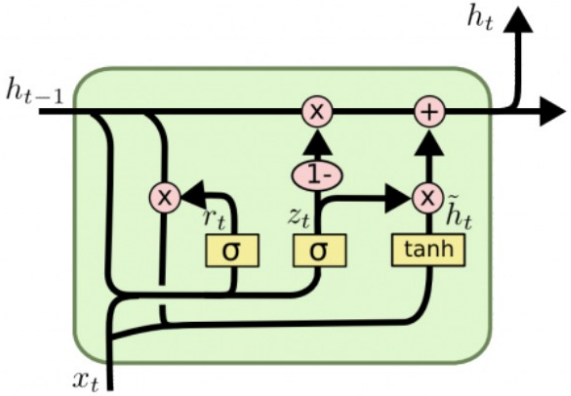
$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Gated Recurrent Unit (GRU)

GRU works just as well as LSTM on many tasks, but is faster to train (**fewer parameters**).

- It merges the hidden state and cell state into **one hidden state h_t**.
- It only has **two gates**:
  - Update Gate (z_t): Combines the "forget" and "input" gates. Decides how much to keep from h_{t-1} vs. how much to add from the new candidate $\tilde{h}_t$.
  - Reset Gate (r_t): Decides how much of the previous memory h_{t-1} to use when calculating the new candidate $\tilde{h}_t$.



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

PyTorch 中的 GRU

```
# Initialize a GRU layer
gru = nn.GRU(input_size=input_size,
             hidden_size=hidden_size,
             num_layers=num_layers,
             batch_first=True)
```

```
# GRU returns (output, h_n)
output, h_n = gru(dummy_input)
```

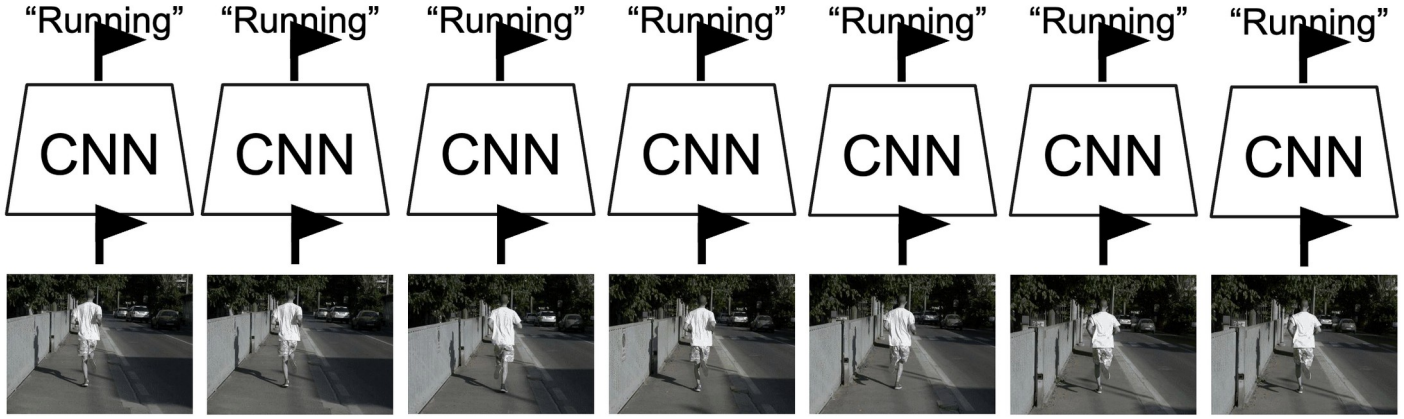
GRU in PyTorch

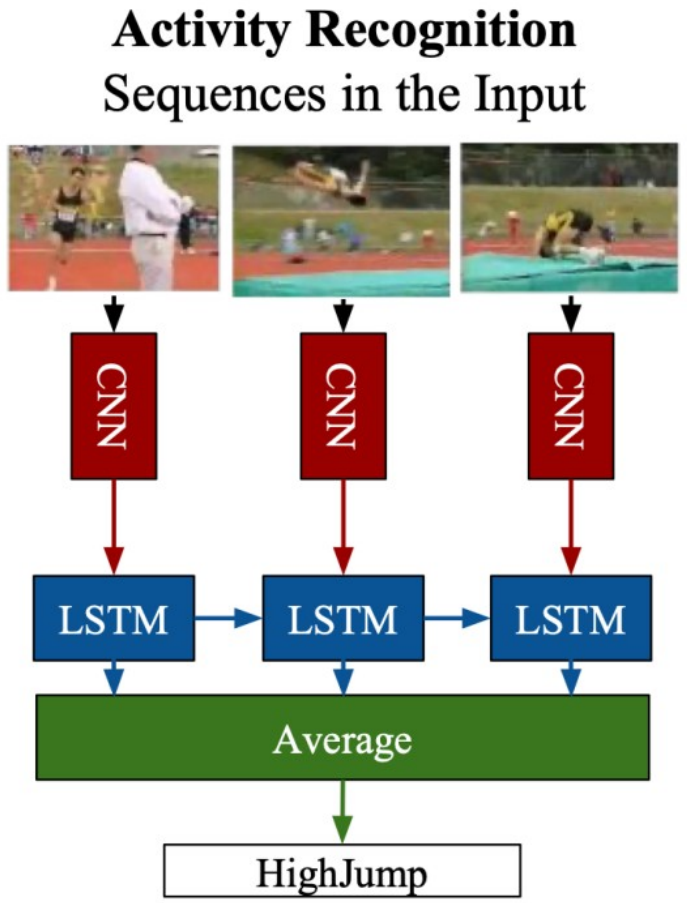
```
# Initialize a GRU layer
gru = nn.GRU(input_size=input_size,
             hidden_size=hidden_size,
             num_layers=num_layers,
             batch_first=True)
```

```
# GRU returns (output, h_n)
output, h_n = gru(dummy_input)
```

CNN+RNN： 时空建模

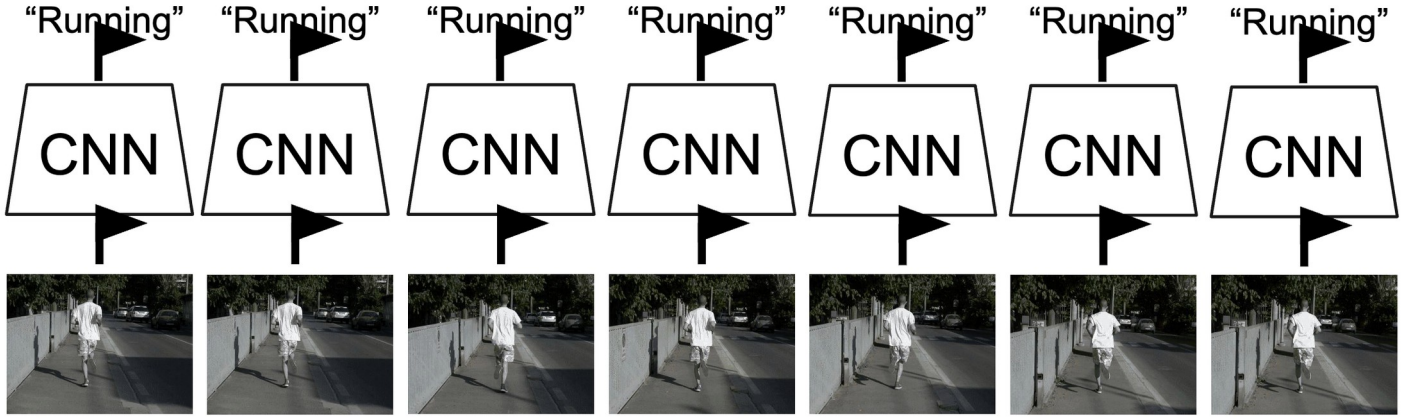
示例：视频理解

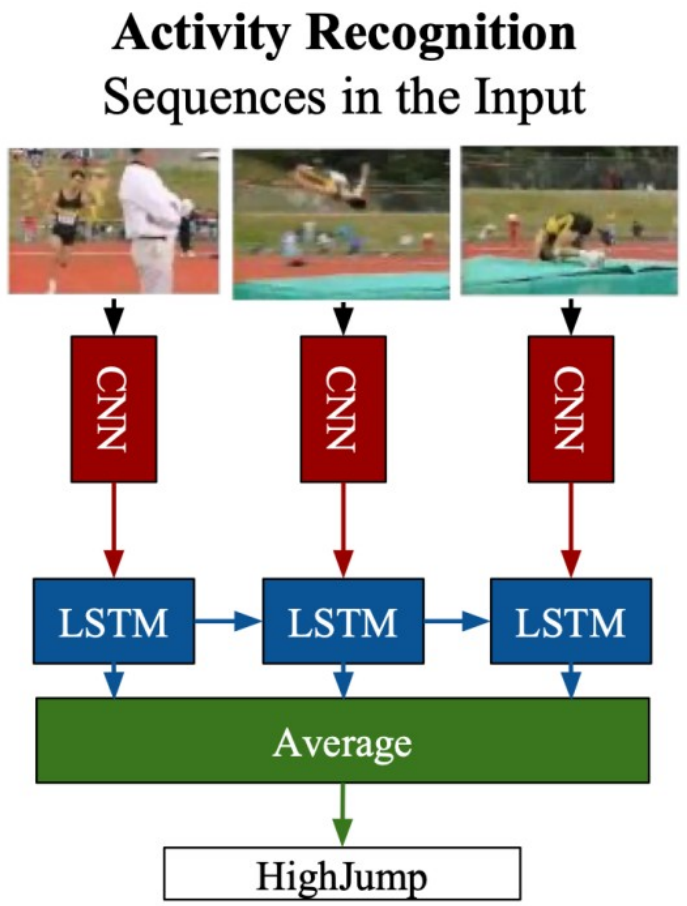




CNN+RNN: Spatio-Temporal Modeling

Example: Video Understanding





PyTorch 中的 CNN+RNN

```python
import torch
import torch.nn as nn


class CNNLSTM(nn.Module):
    def __init__(self, num_classes=10, feature_dim=128,
                 hidden_dim=256, num_layers=1):
        super().__init__()

        # ----- CNN: extract features from each frame -----
        self.cnn = nn.Sequential(
            # Input frame: 3 x 32 x 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),   # -> 32 x 16 x 16

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),   # -> 64 x 8 x 8

            nn.Flatten(),                            # -> 64 * 8 * 8
            nn.Linear(64 * 8 * 8, feature_dim),
            nn.ReLU()
        )

        # ----- LSTM: model temporal dependency across frames -----
        self.lstm = nn.LSTM(
            input_size=feature_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True
        )

        # ----- Classifier: map video feature to class scores -----
        self.classifier = nn.Linear(hidden_dim, num_classes)
```

```python
    def forward(self, x):
        """
        x: [batch, time, channels, height, width]
           e.g., [B, T, 3, 32, 32]
        """

        B, T, C, H, W = x.shape

        # 1. Merge batch and time dimensions
        #    Treat all frames as independent images for CNN
        x = x.view(B * T, C, H, W)
        # x: [B*T, 3, 32, 32]

        # 2. Extract frame-level CNN features
        frame_features = self.cnn(x)
        # frame_features: [B*T, feature_dim]

        # 3. Reshape back to video sequence
        frame_features = frame_features.view(B, T, -1)
        # frame_features: [B, T, feature_dim]

        # 4. Feed frame feature sequence into LSTM
        lstm_outputs, _ = self.lstm(frame_features)
        # lstm_outputs: [B, T, hidden_dim]

        # 5. Average over time
        video_feature = lstm_outputs.mean(dim=1)
        # video_feature: [B, hidden_dim]

        # 6. Predict action class
        logits = self.classifier(video_feature)
        # logits: [B, num_classes]

        return logits
```

CNN+RNN in PyTorch

```python
import torch
import torch.nn as nn


class CNNLSTM(nn.Module):
    def __init__(self, num_classes=10, feature_dim=128,
                 hidden_dim=256, num_layers=1):
        super().__init__()

        # ----- CNN: extract features from each frame -----
        self.cnn = nn.Sequential(
            # Input frame: 3 x 32 x 32
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),   # -> 32 x 16 x 16

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2),   # -> 64 x 8 x 8

            nn.Flatten(),                            # -> 64 * 8 * 8
            nn.Linear(64 * 8 * 8, feature_dim),
            nn.ReLU()
        )

        # ----- LSTM: model temporal dependency across frames -----
        self.lstm = nn.LSTM(
            input_size=feature_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True
        )

        # ----- Classifier: map video feature to class scores -----
        self.classifier = nn.Linear(hidden_dim, num_classes)
```

```python
    def forward(self, x):
        """
        x: [batch, time, channels, height, width]
           e.g., [B, T, 3, 32, 32]
        """

        B, T, C, H, W = x.shape

        # 1. Merge batch and time dimensions
        #    Treat all frames as independent images for CNN
        x = x.view(B * T, C, H, W)
        # x: [B*T, 3, 32, 32]

        # 2. Extract frame-level CNN features
        frame_features = self.cnn(x)
        # frame_features: [B*T, feature_dim]

        # 3. Reshape back to video sequence
        frame_features = frame_features.view(B, T, -1)
        # frame_features: [B, T, feature_dim]

        # 4. Feed frame feature sequence into LSTM
        lstm_outputs, _ = self.lstm(frame_features)
        # lstm_outputs: [B, T, hidden_dim]

        # 5. Average over time
        video_feature = lstm_outputs.mean(dim=1)
        # video_feature: [B, hidden_dim]

        # 6. Predict action class
        logits = self.classifier(video_feature)
        # logits: [B, num_classes]

        return logits
```

随堂练习

任务：用 LSTM 预测下个月的乘客数。

输入： AirPassengers，用已观测的 12 个月预测下一个月

```
import torch
import pandas as pd
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/flights.csv"
s = torch.tensor(pd.read_csv(url).passengers.values, dtype=torch.float32) / 1000
X = torch.stack([s[i:i+12] for i in range(132)]).unsqueeze(-1)
y = s[12:].unsqueeze(-1)
X_train, y_train = X[:108], y[:108]
X_test, y_test = X[108:], y[108:]
```

模型：定义一个 nn.Module

- 使用单层 **nn.LSTM(input_size=1, hidden_size=16, batch_first=True)** 。
- 取其在**最后一个时间步**的输出，再接 **nn.Linear(16,1)** 。

训练与评估

- 全批量（**108 个序列**）， **100 次更新**： **MSELoss** ， **Adam (lr=0.01)** 。
- 每 **20 次更新**打印一次损失。在不计算梯度的情况下评估 **24 个测试序列**。
- 打印预测形状 **[24,1]** 和 **MAE × 1000** （乘客数以千计）。

In-Class Practice

Task: Predict next month's passenger count with an LSTM.

Input: AirPassengers, 12 observed months to predict the next

```
import torch
import pandas as pd
url = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/flights.csv"
s = torch.tensor(pd.read_csv(url).passengers.values, dtype=torch.float32) / 1000
X = torch.stack([s[i:i+12] for i in range(132)]).unsqueeze(-1)
y = s[12:].unsqueeze(-1)
X_train, y_train = X[:108], y[:108]
X_test, y_test = X[108:], y[108:]
```

Model: define an nn.Module

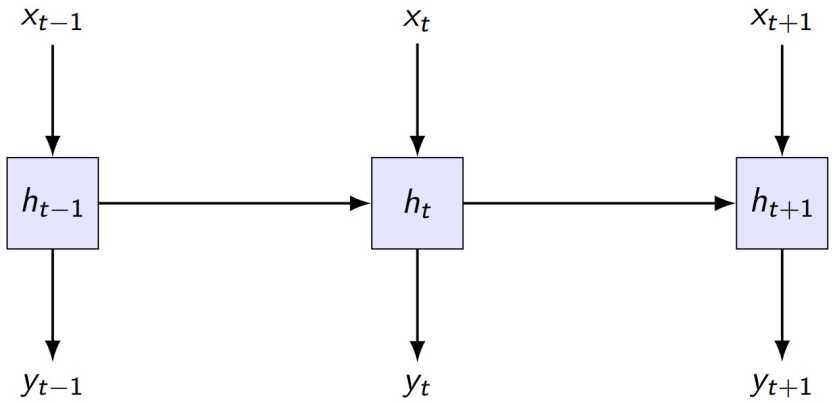
- Use a single-layer **nn.LSTM(input_size=1, hidden_size=16, batch_first=True)**.
- Take its output at the **last time step**, then apply **nn.Linear(16,1)**.

Training and evaluation

- Full batch (**108 sequences**), **100 updates**: **MSELoss**, **Adam (lr=0.01)**.
- Print loss every **20 updates**. Evaluate the **24 test sequences** with no gradients.
- Print prediction shape **[24,1]** and **MAE × 1000** (thousands of passengers).

“Seq-to-Seq”问题

问题：多对多 RNN 每个输入步产生一个输出，因此输入与输出长度相同。



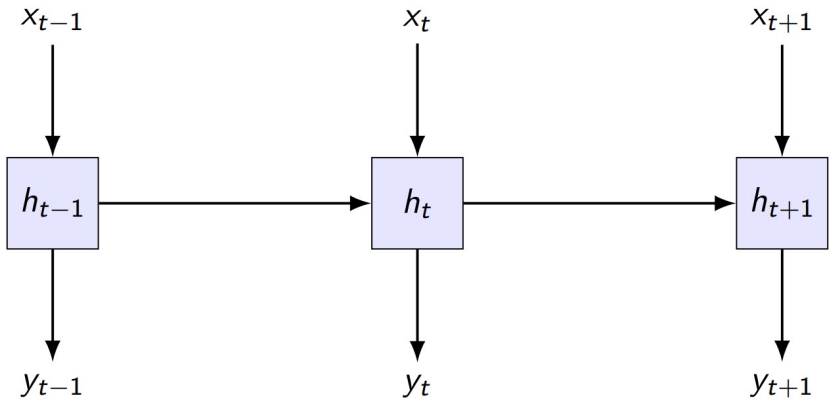
示例任务：“What is your name?”（4 个词）回答：“James Bond”（2 个词）

长度不一致，需要新的架构。

解决方案：编码器 - 解码器模型

The"Seq-to-Seq" Problem

Problem: The many-to-many RNN produces one output for each input step, so the input and output lengths are the same.



Example Task: “What is your name?” (4 words) Answer: “James Bond" (2 words)
The lengths don’t match. We need a new architecture.

Solution: Encoder-Decoder Model

编码器 - 解码器架构

可以使用两个 RNN。该模型允许输入与输出长度不同。

编码器（读者）：“多对一” RNN。
- 它读入整个输入句子（“What is your name?”）。
- 它忽略所有输出，只**保留最终隐藏状态 $\mathbf{h}_N$**。
- 这个 h_N 就是“上下文向量”或“思想向量”——整个输入句子的概括。

解码器（写者）：“一对多” RNN。
- 其初始隐藏状态设为编码器的最终 h_N。
- 随后在该初始“上下文”条件下逐词生成输出句子。

这一直是 Seq-to-Seq 任务的标准架构，直到…… Transformer 出现！

Encoder-Decoder Architecture

We can use two RNNs. This model allows the input and output to have different lengths.

Encoder (Reader): A "Many-to-One" RNN.

- It reads the entire input sentence ("What is your name?").
- It ignores all outputs, and just **keeps the final hidden state h_N**.
- This h_N is the "context vector" or "thought vector" – a summary of the whole input sentence.

Decoder (Writer): A "One-to-Many" RNN.

- Its initial hidden state is set to the Encoder's final h_N.
- It then generates the output sentence word-by-word, conditioned on that initial "context".

This was the standard architecture for Seq-to-Seq task until… Transformers!

PyTorch 中简单的 RNN 编码器 - 解码器

```python
import torch
import torch.nn as nn


class SimpleSeq2Seq(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size,
                 embed_dim=256, hidden_dim=512):
        super().__init__()

        # ----- Encoder -----
        # Convert source token IDs into vectors
        self.src_embedding = nn.Embedding(src_vocab_size, embed_dim)

        # Read the source sequence and produce a final hidden state
        self.encoder_rnn = nn.GRU(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        # ----- Decoder -----
        # Convert target token IDs into vectors
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, embed_dim)

        # Generate the target sequence, starting from encoder_hidden
        self.decoder_rnn = nn.GRU(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        # Map decoder hidden states to vocabulary scores
        self.output_layer = nn.Linear(hidden_dim, tgt_vocab_size)
```

```python
def forward(self, src_ids, tgt_input_ids):
    """
    src_ids:        [batch, src_len]
    tgt_input_ids:  [batch, tgt_len]

    returns:
    logits:         [batch, tgt_len, tgt_vocab_size]
    """

    # 1. Encode source sentence
    src_emb = self.src_embedding(src_ids)
    # src_emb: [batch, src_len, embed_dim]

    encoder_outputs, encoder_hidden = self.encoder_rnn(src_emb)
    # encoder_outputs: [batch, src_len, hidden_dim]
    # encoder_hidden:  [1, batch, hidden_dim]

    # 2. Decode target sentence
    tgt_emb = self.tgt_embedding(tgt_input_ids)
    # tgt_emb: [batch, tgt_len, embed_dim]

    decoder_outputs, decoder_hidden = self.decoder_rnn(
        tgt_emb,
        encoder_hidden
    )
    # decoder_outputs: [batch, tgt_len, hidden_dim]
    # decoder_hidden:  [1, batch, hidden_dim]

    # 3. Predict target tokens
    logits = self.output_layer(decoder_outputs)
    # logits: [batch, tgt_len, tgt_vocab_size]

    return logits
```

A Simple RNN Encoder-Decoder in PyTorch

```python
import torch
import torch.nn as nn


class SimpleSeq2Seq(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size,
                 embed_dim=256, hidden_dim=512):
        super().__init__()

        # ----- Encoder -----
        # Convert source token IDs into vectors
        self.src_embedding = nn.Embedding(src_vocab_size, embed_dim)

        # Read the source sequence and produce a final hidden state
        self.encoder_rnn = nn.GRU(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        # ----- Decoder -----
        # Convert target token IDs into vectors
        self.tgt_embedding = nn.Embedding(tgt_vocab_size, embed_dim)

        # Generate the target sequence, starting from encoder_hidden
        self.decoder_rnn = nn.GRU(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        # Map decoder hidden states to vocabulary scores
        self.output_layer = nn.Linear(hidden_dim, tgt_vocab_size)
```

```python
def forward(self, src_ids, tgt_input_ids):
    """
    src_ids:        [batch, src_len]
    tgt_input_ids:  [batch, tgt_len]

    returns:
    logits:         [batch, tgt_len, tgt_vocab_size]
    """

    # 1. Encode source sentence
    src_emb = self.src_embedding(src_ids)
    # src_emb: [batch, src_len, embed_dim]

    encoder_outputs, encoder_hidden = self.encoder_rnn(src_emb)
    # encoder_outputs: [batch, src_len, hidden_dim]
    # encoder_hidden:  [1, batch, hidden_dim]

    # 2. Decode target sentence
    tgt_emb = self.tgt_embedding(tgt_input_ids)
    # tgt_emb: [batch, tgt_len, embed_dim]

    decoder_outputs, decoder_hidden = self.decoder_rnn(
        tgt_emb,
        encoder_hidden
    )
    # decoder_outputs: [batch, tgt_len, hidden_dim]
    # decoder_hidden:  [1, batch, hidden_dim]

    # 3. Predict target tokens
    logits = self.output_layer(decoder_outputs)
    # logits: [batch, tgt_len, tgt_vocab_size]

    return logits
```



语言建模与 Transformer



Language Modeling and Transformer

什么是语言建模?

- **语言建模**：刻画人类语言统计特性与模式的任务。
- 其核心是回答一个问题：“接下来是什么？”

示例

给定上下文：“The students opened their...”

系统须给出概率：

- books （概率高）
- laptops （概率中等）
- bananas （概率极低）

What is Language Modeling?

- **Language Modeling**: The task of capturing the statistical properties and patterns of human language.
- At its core, it's about answering the question: "What comes next?"

Example

Given the context: "The students opened their…"

A system must assign probabilities:

- books (high probability)
- laptops (medium probability)
- bananas (very low probability)

什么是语言模型?

- 语言模型（LM）是执行语言建模任务的系统、模型或算法。
- 为此，系统必须学习所有可能序列上的概率分布。

What is a Language Model?

- A language Model (LM) is the system, model, or algorithm that performs the Language Modeling Task.
- To do this, the system must learn a probability distribution over all possible sequences.

什么是语言模型?

- **Token （词元）**：语言模型的基本构成单位（符号、词或子词）
- **语言模型定义** P：所有可能 token 序列上的概率分布
- **核心任务**：模型须在给定前文的条件下计算下一个 token 的条件概率。

- **序列概率**：整句的概率等于这 $P(y_t|y^{t-1})$ 乘积

$$P(y^T) = \prod_{t=1}^{T} P(y_t|y^{t-1})$$

What is a Language Model?

- **Tokens**: The basic building blocks (symbols, words, or subwords) of LMs
- **A language model defines** P: A probability distribution over all possible sequences of tokens
- **Core Task**: The model must compute the conditional probability of the next token given the preceding history.

$$P(y_t|y^{t-1})$$

- **Sequence Probability**: The probability of an entire sentence is the product of these conditional probabilities

$$P(y^T) = \prod_{t=1}^{T} P(y_t|y^{t-1})$$

如何学习语言模型？（MLE）

假设类（H）： 先选择一种模型架构，它定义了一组模型 h_θ。
训练数据（y^T）：大规模文本语料。

学习目标：最大似然估计（MLE）
- 找到使观测到训练数据的概率最大的参数

$$\hat{\theta} := arg\ max_{\theta \in \Theta} \prod_{t=1}^{T} h_\theta(y^{t-1})[y_t]$$

表示模型预测的条件分布

- **实践中（对数损失）：** 这等价于最小化负对数似然（对数损失）

$$\hat{\theta} = arg\ min_{\theta \in \Theta} \sum_{t=1}^{T} -log\ h_\theta(y^{t-1})[y_t]$$

How is an LM Learned? (MLE)

Hypothesis Class (H): We first choose a model architecture, which defines a set of models h_θ .

Training Data (y^T): A large corpus of text.

Learning Objective: Maximum Likelihood Estimation (MLE)

- Find the parameters that maximize the probability of observing the training data

$$\hat{\theta} := arg\ max_{\theta \in \Theta} \prod_{t=1}^{T} h_\theta(y^{t-1})[y_t]$$

represents the model's predicted conditional distribution

- **In Practice (Log Loss):** This is equivalent to minimizing the Negative Log-Likelihood (Logarithmic Loss)

$$\hat{\theta} = arg\ min_{\theta \in \Theta} \sum_{t=1}^{T} -log\ h_\theta(y^{t-1})[y_t]$$

PyTorch 中的语言模型训练

准备

```
import torch
import torch.nn as nn
import torch.optim as optim
# 1. Hypothesis Class (e.g., a Transformer)
# model = MyTransformerModel(vocab_size, ...)
# h_theta
model = ... # (Instantiate our model)

# 2. Training Data
# context = "The students opened" -> [5, 27, 89]
# target  = "their"               -> [101]
context_ids = torch.tensor([[5, 27, 89]])  # y^(t-1)
target_ids  = torch.tensor([101])          # y_t

# 3. Learning Objective (Negative Log-Likelihood)
# nn.CrossEntropyLoss is the key!
# It directly implements: -log h_theta(y^(t-1))[y_t]
loss_function = nn.CrossEntropyLoss()
# 4. Optimization Algorithm (e.g., Adam)
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

LM Training in PyTorch

Setup

```
import torch
import torch.nn as nn
import torch.optim as optim
# 1. Hypothesis Class (e.g., a Transformer)
# model = MyTransformerModel(vocab_size, ...)
# h_theta
model = ... # (Instantiate our model)

# 2. Training Data
# context = "The students opened" -> [5, 27, 89]
# target  = "their"               -> [101]
context_ids = torch.tensor([[5, 27, 89]])  # y^(t-1)
target_ids  = torch.tensor([101])          # y_t

# 3. Learning Objective (Negative Log-Likelihood)
# nn.CrossEntropyLoss is the key!
# It directly implements: -log h_theta(y^(t-1))[y_t]
loss_function = nn.CrossEntropyLoss()
# 4. Optimization Algorithm (e.g., Adam)
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

PyTorch 中的语言模型训练

训练步骤

```
# --- A Single Training Step ---
# Zero out gradients from last step
optimizer.zero_grad()

# Forward pass: Get predictions (logits)
# Logits = h_theta(y^(t-1))
logits = model(context_ids)
# E.g., logits shape: [1, vocab_size]

# Compute the Log Loss
# This is the core of MLE
loss = loss_function(logits, target_ids)

# Backward pass: Compute gradients
loss.backward()
# Update model parameters (theta)
optimizer.step()
```

LM Training in PyTorch

Training Step

```
# --- A Single Training Step ---
# Zero out gradients from last step
optimizer.zero_grad()

# Forward pass: Get predictions (logits)
# Logits = h_theta(y^(t-1))
logits = model(context_ids)
# E.g., logits shape: [1, vocab_size]

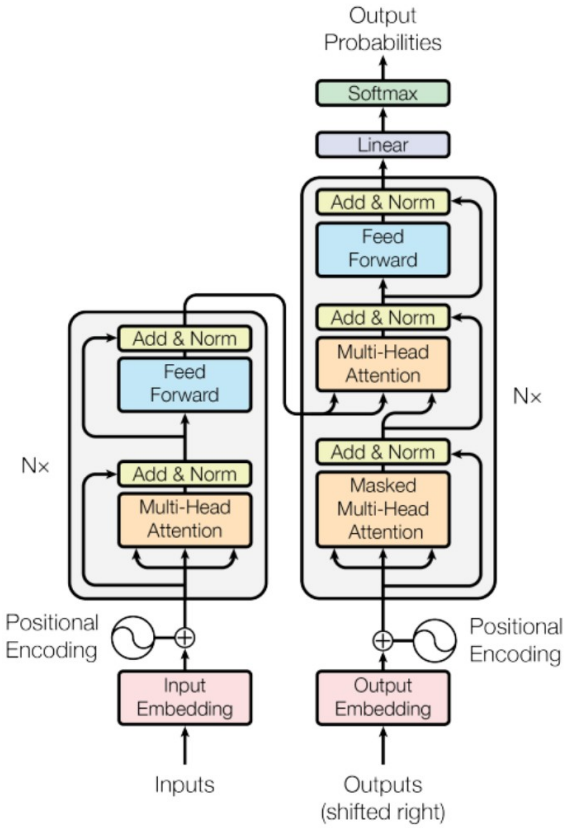
# Compute the Log Loss
# This is the core of MLE
loss = loss_function(logits, target_ids)

# Backward pass: Compute gradients
loss.backward()
# Update model parameters (theta)
optimizer.step()
```

最先进的语言模型： Transformer 模型

学习的关键在于为假设类 H 选择好的架构。

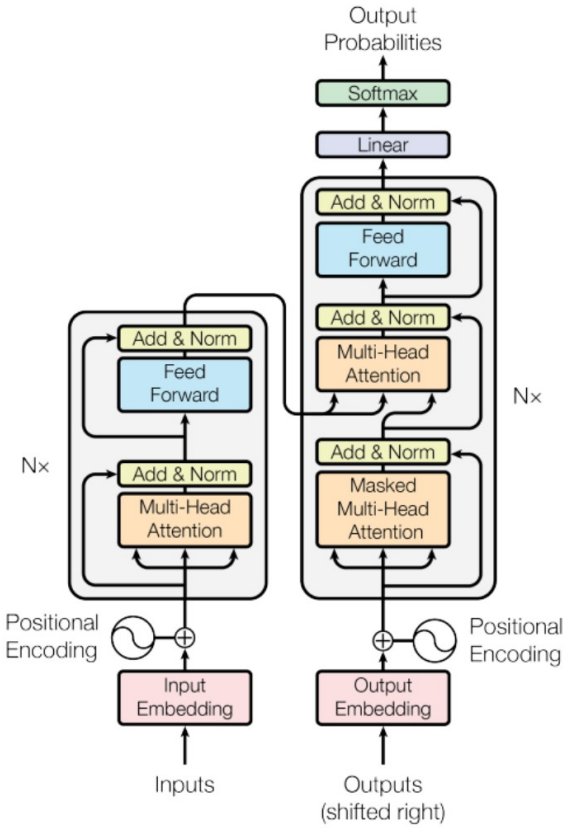
Transformer 模型：使用**自注意力**并行处理序列。



SOTA LM: Transformer Models

The key to learning is choosing a good architecture for the hypothesis class H.

Transformer Models: Use **self-attention** to process sequences in parallel.

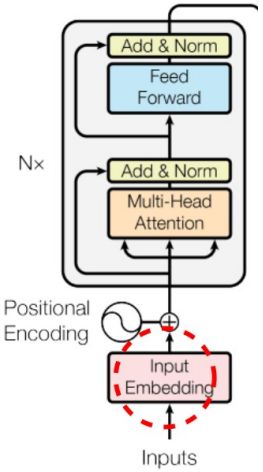


组件 1：输入嵌入

这是网络的第一个可学习层，把离散 token 转换为连续向量。

词表 (Y)：所有可能 token 的集合。

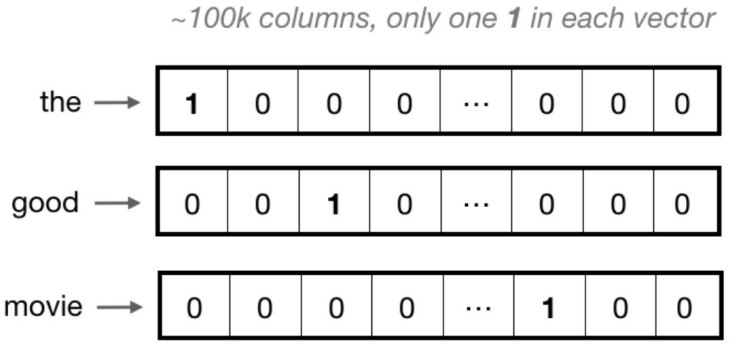
- 例如 GPT-3 中 $|Y| \sim 50,000$



Large generative models (e.g., large language models, diffusion models) have shown remarkable performance, but they require a massive amount of computational resources. To make them more accessible, it is crucial to improve their efficiency.This course will introduce efficient AI computing techniques that enable powerful deep learning applications on resource-constrained devices. Topics include model compression, pruning, quantization, neural architecture search, distributed training, data/model parallelism, gradient compression, and on-device fine-tuning. It also introduces application-specific acceleration techniques for large language models, diffusion models, video recognition, and point cloud. This course will also cover topics about quantum machine learning. Students will get hands-on experience deploying large language models (e.g., LLaMA 2) on a laptop.

TEXT TOKEN IDS

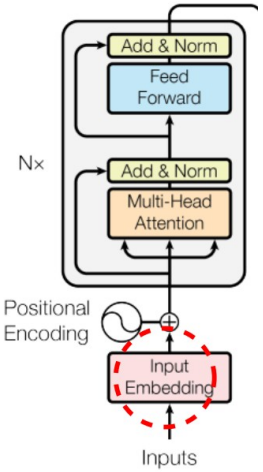
分词器把一个词映射为一个或多个 token



独热编码：把每个词表示为一个向量，其长度等于词表大小；向量的每一列代表词表中的一个可能的词。

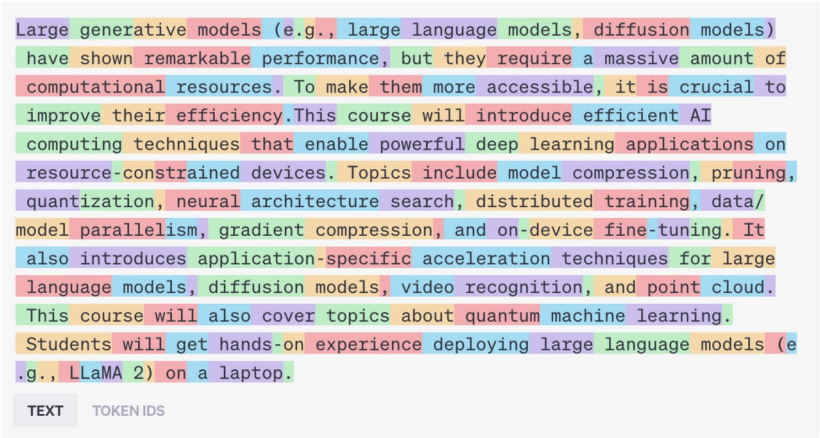
Component 1: Input Embedding

This is the first learnable layer of the network, converting discrete tokens into continuous vectors.

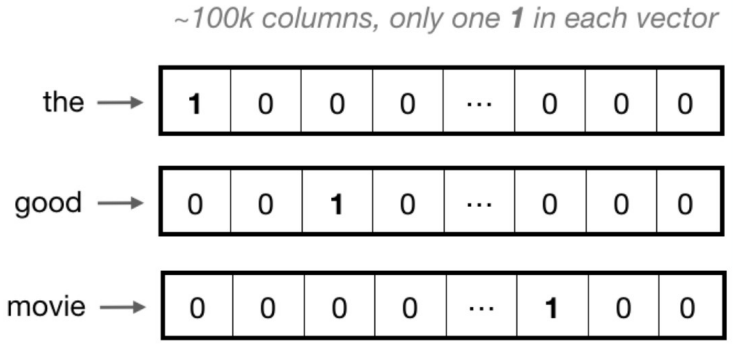


Vocabulary (Y): The set of all possible tokens.

- e.g., In GPT-3, |Y| ∼ 50, 000



A tokenizer maps a word to one/multiple tokens



One-hot Encoding: Representing each word as a vector with as many values as there are words in the vocabulary. Each column in a vector represents one possible word in a vocabulary.

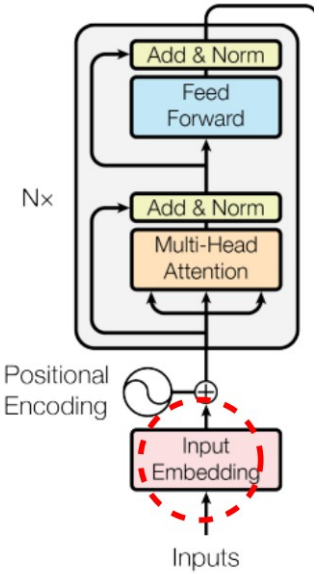
组件 1：输入嵌入

这是网络的第一个可学习层，把离散 token 转换为连续向量。

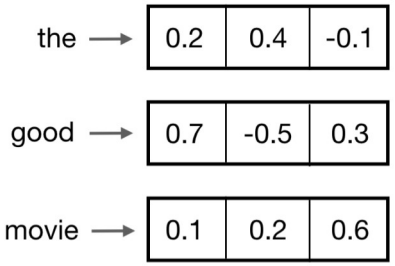
词表 (Y)：所有可能 token 的集合。
- 例如 GPT-3 中 $|Y| \sim 50,000$

嵌入矩阵 (M)：一个大型可学习矩阵 $M \in \mathbb{R}^{d \times |Y|}$.
- d 为嵌入维度（如 GPT-3 中 $d = 12,288$）。

映射：
- 每个 token y （表示为独热向量 e_y）被映射为稠密向量 x_y。

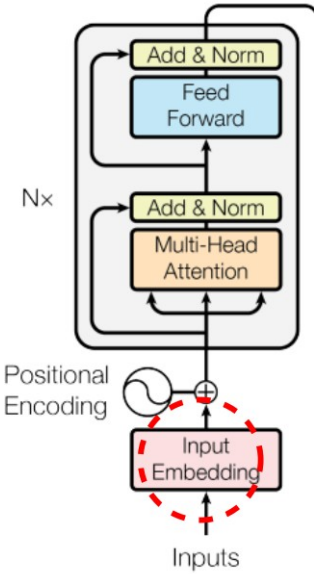


$$x_y := Me_y \in \mathbb{R}^d$$



Component 1: Input Embedding

This is the first learnable layer of the network, converting discrete tokens into continuous vectors.



Vocabulary (Y): The set of all possible tokens.

- e.g., In GPT-3, $|Y| \sim 50,000$

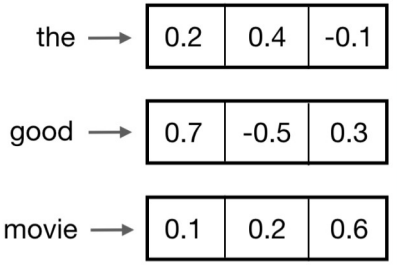
Embedding Matrix (M): A large, learnable matrix $M \in R^{d \times |Y|}$.

- d is the embedding dimension (e.g., d = 12, 288 for GPT-3).

Mapping:

- Each token y (represented as a one-hot vector e_y) is mapped to a dense vector x_y.

$$x_y := Me_y \in \mathbb{R}^d$$



PyTorch 中的 Token 嵌入： nn.Embedding

```
import torch
import torch.nn as nn
import math
# --- Model Hyperparameters ---
vocab_size = 50000  # |Y|
d_model = 768       # d (embedding dimension)
max_seq_len = 1024  # Maximum sequence length
# --- Token Embedding Layer ---
# This is our learnable matrix M
# It's a simple lookup table: index -> vector
token_embedding_layer = nn.Embedding(vocab_size, d_model)
# --- Example Usage ---
# Batch of 2 sentences, 5 tokens each
# (e.g., [[ "The", "cat", "sat", "on", "the" ],
#         [ "A", "dog", "ran", ".", "<pad>" ]])
input_ids = torch.randint(0, vocab_size, (2, 5))
# input_ids.shape: [batch_size=2, seq_len=5]
# Pass token IDs through the embedding layer
# X = M * e_y
token_embeddings = token_embedding_layer(input_ids)
# token_embeddings.shape: [2, 5, d_model=768]
```

Token Embedding in PyTorch: nn.Embedding

```python
import torch
import torch.nn as nn
import math
# --- Model Hyperparameters ---
vocab_size = 50000  # |Y|
d_model = 768       # d (embedding dimension)
max_seq_len = 1024  # Maximum sequence length
# --- Token Embedding Layer ---
# This is our learnable matrix M
# It's a simple lookup table: index -> vector
token_embedding_layer = nn.Embedding(vocab_size, d_model)
# --- Example Usage ---
# Batch of 2 sentences, 5 tokens each
# (e.g., [[ "The", "cat", "sat", "on", "the" ],
#         [ "A", "dog", "ran", ".", "<pad>" ]])
input_ids = torch.randint(0, vocab_size, (2, 5))
# input_ids.shape: [batch_size=2, seq_len=5]
# Pass token IDs through the embedding layer
# X = M * e_y
token_embeddings = token_embedding_layer(input_ids)
# token_embeddings.shape: [2, 5, d_model=768]
```

组件 2：位置编码

仅靠 token 嵌入和自注意力无法捕捉词序。

解决方案：位置编码

- 为每个位置 i 定义一个向量 $p_i \in R^d$。
- 该 P 矩阵被加到输入 token 嵌入 X 上

$$X' = X + P \in \mathbb{R}^{t \times d}, t = \text{seq_len}$$

Component 2: Positional Encoding

Token embeddings and self-attention alone do not capture word order.

Solution: Positional Encoding

- A vector $p_i \in R^d$ is defined for each position i.
- This P matrix is added to the input token embeddings X

$$X' = X + P \in \mathbb{R}^{t \times d}, t = \text{seq_len}$$

PyTorch 中的位置编码

```python
import torch
import torch.nn as nn

# --- Model Hyperparameters ---
vocab_size = 50000      # vocabulary size
d_model = 768           # embedding dimension
max_seq_len = 1024      # maximum sequence length

# --- Token Embedding Layer ---
# Converts token IDs into token vectors
token_embedding_layer = nn.Embedding(vocab_size, d_model)

# --- Positional Embedding Layer ---
# Converts position IDs into position vectors
position_embedding_layer = nn.Embedding(max_seq_len, d_model)

# --- Example Usage ---
# Batch of 2 sentences, 5 tokens each
# e.g., [["The", "cat", "sat", "on", "the"],
#        ["A", "dog", "ran", ".", "<pad>"]]
input_ids = torch.randint(0, vocab_size, (2, 5))
# input_ids.shape: [batch_size=2, seq_len=5]

batch_size, seq_len = input_ids.shape

# Create position IDs: [0, 1, 2, 3, 4]
position_ids = torch.arange(seq_len)
# position_ids.shape: [seq_len]

# Expand to match the batch dimension
position_ids = position_ids.unsqueeze(0).expand(batch_size, seq_len)
# position_ids.shape: [batch_size=2, seq_len=5]

# Get token embeddings
token_embeddings = token_embedding_layer(input_ids)
# token_embeddings.shape: [2, 5, d_model=768]

# Get positional embeddings
position_embeddings = position_embedding_layer(position_ids)
# position_embeddings.shape: [2, 5, d_model=768]

# Add them together
input_embeddings = token_embeddings + position_embeddings
# input_embeddings.shape: [2, 5, d_model=768]
```

Positional Encoding in PyTorch

```python
import torch
import torch.nn as nn

# --- Model Hyperparameters ---
vocab_size = 50000      # vocabulary size
d_model = 768           # embedding dimension
max_seq_len = 1024      # maximum sequence length

# --- Token Embedding Layer ---
# Converts token IDs into token vectors
token_embedding_layer = nn.Embedding(vocab_size, d_model)

# --- Positional Embedding Layer ---
# Converts position IDs into position vectors
position_embedding_layer = nn.Embedding(max_seq_len, d_model)

# --- Example Usage ---
# Batch of 2 sentences, 5 tokens each
# e.g., [["The", "cat", "sat", "on", "the"],
#        ["A", "dog", "ran", ".", "<pad>"]]
input_ids = torch.randint(0, vocab_size, (2, 5))
# input_ids.shape: [batch_size=2, seq_len=5]

batch_size, seq_len = input_ids.shape

# Create position IDs: [0, 1, 2, 3, 4]
position_ids = torch.arange(seq_len)
# position_ids.shape: [seq_len]

# Expand to match the batch dimension
position_ids = position_ids.unsqueeze(0).expand(batch_size, seq_len)
# position_ids.shape: [batch_size=2, seq_len=5]

# Get token embeddings
token_embeddings = token_embedding_layer(input_ids)
# token_embeddings.shape: [2, 5, d_model=768]

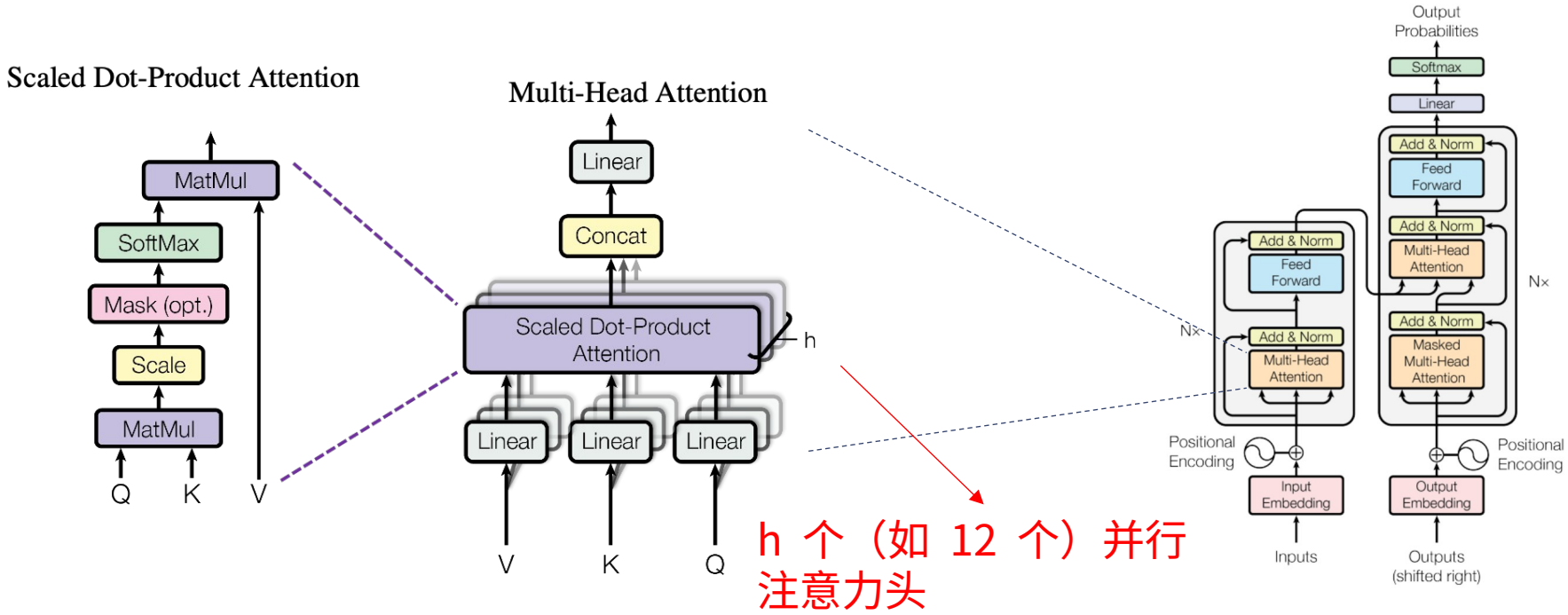
# Get positional embeddings
position_embeddings = position_embedding_layer(position_ids)
# position_embeddings.shape: [2, 5, d_model=768]

# Add them together
input_embeddings = token_embeddings + position_embeddings
# input_embeddings.shape: [2, 5, d_model=768]
```

组件 3：多头注意力（MHA）

MHA 是 Transformer 的核心注意力机制

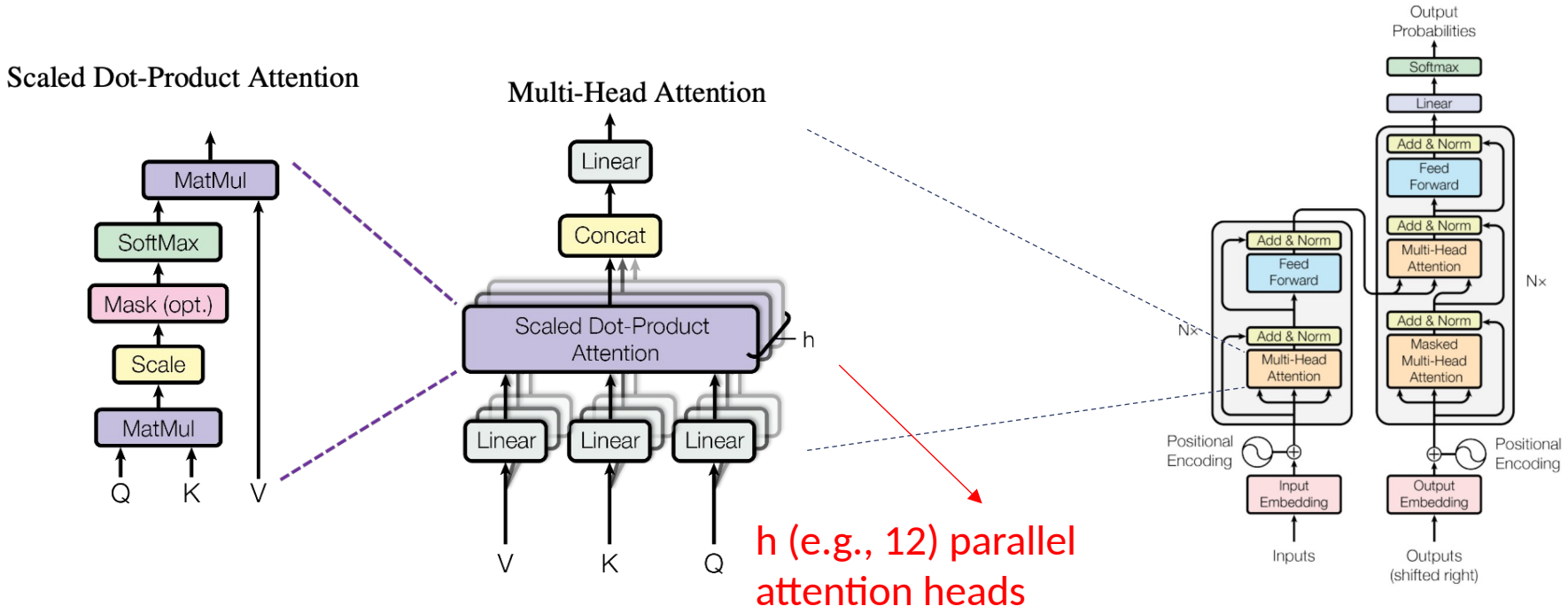
- **并行**：高度可并行，在现代硬件（GPU）上效率高
- 要理解完整的 MHA，先要理解其基本构件：缩放点积注意力（即“单个头”）



Component 3: Multi-Head Attention (MHA)

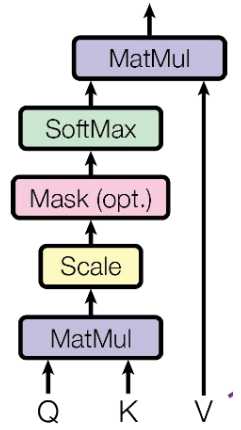
MHA is the core attention mechanism of the Transformer

- **Parallel**: It's highly parallelizable, making it efficient for modern hardware (GPUs)
- To understand the full MHA, we must first understand its basic building block: the Scaled Dot-Product Attention (i.e., a "single head")



缩放点积注意力（“头”）

单个注意力头 i 的核心逻辑



Project: Create Query, Key, and Value matrices from the input $X \in \mathbb{R}^{t \times d}$ (t=seq_len, d=model_dim) (after position embedding) using learnable weights $W_q^i, W_k^i, W_v^i \in \mathbb{R}^{d \times d'}$ (d'=head_dim = d/h).

- $Q_i := XW_q^i \in [t, d']$
- $K_i := XW_k^i \in [t, d']$
- $V_i := XW_v^i \in [t, d']$

Calculate Scores: Compute scaled dot-product attention scores. “How much does each token ‘query’ `match` each ‘key’?”

$$S_i := \frac{(Q_i K_i^\top)}{\sqrt{d'}} \in \mathbb{R}^{t \times t}$$

Get Weights (Softmax): Apply softmax row-wise to get attention weights (probabilities).

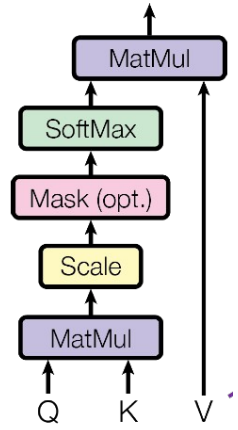
$$P_i := softmax(S_i) \in \mathbb{R}^{t \times t}$$

Get Output: Weighted sum of Values based on attention weights.

$$X_i := P_i V_i \in \mathbb{R}^{t \times d'}$$

Scaled Dot-Product Attention ("Head")

The core logic of a single attention head *i*



Project: Create Query, Key, and Value matrices from the input $X \in \mathbb{R}^{t \times d}$ (t=seq_len, d=model_dim) (after position embedding) using learnable weights $W_q^i, W_k^i, W_v^i \in \mathbb{R}^{d \times d'}$ (d'=head_dim = d/h).

- $Q_i := XW_q^i \in [t, d']$
- $K_i := XW_k^i \in [t, d']$
- $V_i := XW_v^i \in [t, d']$

Calculate Scores: Compute scaled dot-product attention scores. "How much does each token 'query' `match` each 'key'?"

$$S_i := \frac{(Q_i K_i^\top)}{\sqrt{d'}} \in \mathbb{R}^{t \times t}$$

Get Weights (Softmax): Apply softmax row-wise to get attention weights (probabilities).

$$P_i := softmax(S_i) \in \mathbb{R}^{t \times t}$$

Get Output: Weighted sum of Values based on attention weights.

$$X_i := P_i V_i \in \mathbb{R}^{t \times d'}$$

为什么要缩放点积?

问题:

- 点积得分可能变得很大，尤其是维度 d′ 很大时。
- 得分差异大会使 softmax 输出非常接近 0 或 1。
  - ‘softmax([1, 2])’ → ‘[0.27, 0.73]’
  - ‘softmax([10, 80])’ → ‘[0.0, 1.0]’
- softmax 输出接近 0 或 1 时，梯度变得极小，训练变得困难。

解决方案:

- 若 q 和 k 的均值为 0、方差为 1，则其点积的方差为 d′。
- 将点积除以标准差 $\sqrt{d'}$，使方差回到 1 左右，让 softmax 的输入保持在合理尺度。

Why Scale the Dot Products?

Problem:

- The dot-product scores can become very large, especially when the dimension d′ is large.
- Large score differences make the ‘softmax’ outputs very close to 0 or 1.
  - ‘softmax([1, 2])’ → ‘[0.27, 0.73]’
  - ‘softmax([10, 80])’ → ‘[0.0, 1.0]’
- When softmax outputs are very close to 0 or 1, the gradients become extremely small, making training difficult.

Solution:

- If q and k have mean 0 and variance 1, their dot product has variance d′.
- Divide the dot product by the standard deviation $\sqrt{d'}$ to bring the variance back to around 1, keeping the inputs to ‘softmax’ at a reasonable scale.

PyTorch 中的缩放点积注意力

```python
import torch.nn.functional as F
import math
def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q, K, V shape: [Batch, Num_Heads, Seq_Len, Head_Dim]
    # 1. Get head dimension
    d_k = Q.size(-1)
    # 2. Calculate Scores: (B, H, T, T)
    #    (B, H, T, D) @ (B, H, D, T) -> (B, H, T, T)
    scores = torch.matmul(Q, K.transpose(-2, -1))
    # 3. Scale: (The focus of the last slide)
    scores = scores / math.sqrt(d_k)
    # 4. (Optional) Apply Mask
    #    (e.g., for padding or for decoder look-ahead)
    if mask is not None:
        # Fills -infinity where mask is 1
        scores = scores.masked_fill(mask == 1, -1e9)
    # 5. Get Weights (Softmax)
    weights = F.softmax(scores, dim=-1)
    # 6. Get Output (Weighted sum of V)
    #    (B, H, T, T) @ (B, H, T, D) -> (B, H, T, D)
    output = torch.matmul(weights, V)
    return output, weights
```

Scaled Dot-Product Attention in PyTorch

```python
import torch.nn.functional as F
import math
def scaled_dot_product_attention(Q, K, V, mask=None):
    # Q, K, V shape: [Batch, Num_Heads, Seq_Len, Head_Dim]
    # 1. Get head dimension
    d_k = Q.size(-1)
    # 2. Calculate Scores: (B, H, T, T)
    #    (B, H, T, D) @ (B, H, D, T) -> (B, H, T, T)
    scores = torch.matmul(Q, K.transpose(-2, -1))
    # 3. Scale: (The focus of the last slide)
    scores = scores / math.sqrt(d_k)
    # 4. (Optional) Apply Mask
    #    (e.g., for padding or for decoder look-ahead)
    if mask is not None:
        # Fills -infinity where mask is 1
        scores = scores.masked_fill(mask == 1, -1e9)
    # 5. Get Weights (Softmax)
    weights = F.softmax(scores, dim=-1)
    # 6. Get Output (Weighted sum of V)
    #    (B, H, T, T) @ (B, H, T, D) -> (B, H, T, D)
    output = torch.matmul(weights, V)
    return output, weights
```

随堂练习

问题：

- 我们按批处理句子，而句子长度各不相同。
- 用 pad_token （如 ID 0 ）填充较短的句子，使批中所有张量大小相同。
- 不能让模型“关注”这些填充 token 。

任务：

- 我们的 scaled_dot_product_attention 函数会在 mask == 1 处进行填充。
- 因此需要一个掩码：真实 token 处为 False （或 0 ），填充 token 处为 True （或 1 ）。
- 它还须具有可广播的形状： [B, 1, 1, T] 。

```
# Batch of 2 sentences, Seq length
      is 4, 0 is the <pad> token
input_ids = torch.tensor([
    [5, 8, 12, 0],  #"The cat sat"
    [7, 2, 0, 0]   #"A dog"
])
# Shape: [B=2, T=4]
```

In-Class Practice

Problem:

- We process sentences in batches. Sentences have different lengths.
- We pad short sentences with a ‘pad_token’ (e.g., ID 0) so all tensors in the batch are the same size.
- We must not let the model “attend” to these padding tokens..

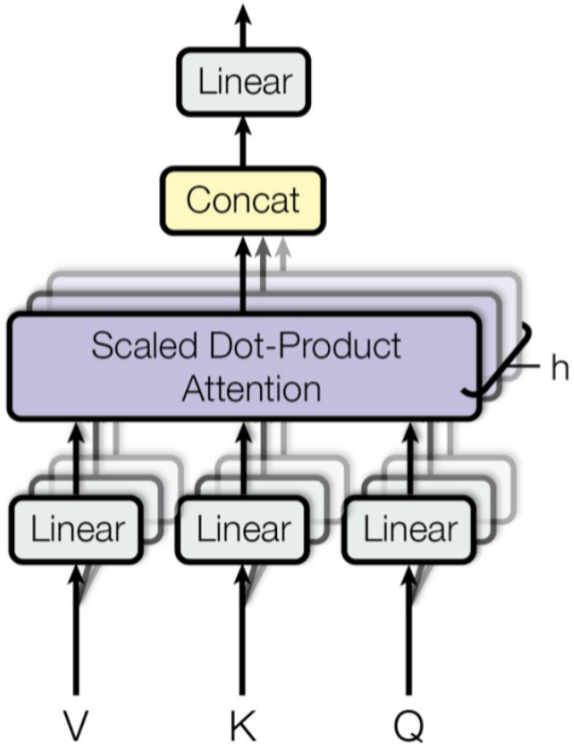
Task:

- Our ‘scaled_dot_product_attention’ function fills where ‘mask == 1’.
- This means we need a mask that is False (or 0) for real tokens and True (or 1) for pad tokens.
- It must also have the broadcastable shape: [B, 1, 1, T].

```
# Batch of 2 sentences, Seq length
      is 4, 0 is the <pad> token
input_ids = torch.tensor([
    [5, 8, 12, 0],  #"The cat sat"
    [7, 2, 0, 0]   #"A dog"
])
# Shape: [B=2, T=4]
```

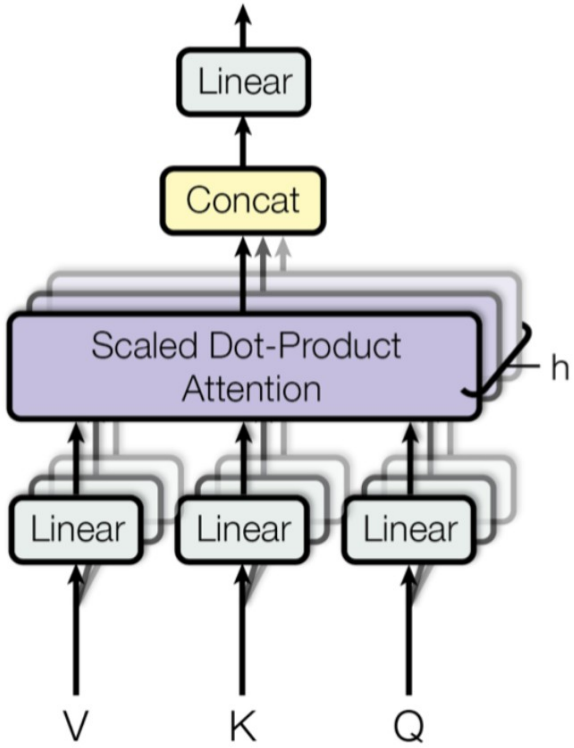
组件 3： 多头注意力（MHA）

Linearly project $\boldsymbol{X}$ to get h different sets of $(\boldsymbol{Q_i}, \boldsymbol{K_i}, \boldsymbol{V_i})$.
Run h independent SDPA heads in parallel.
This yields h separate output matrices $(\boldsymbol{X}_1, \ldots, \boldsymbol{X_h})$.
Concatenate all h outputs: $Concat(\boldsymbol{X}_1, \ldots, \boldsymbol{X_h})$.
Apply a final linear projection $\boldsymbol{W^O}$.



Component 3: Multi-Head Attention (MHA)

Linearly project $\boldsymbol{X}$ to get h different sets of $(\boldsymbol{Q_i}, \boldsymbol{K_i}, \boldsymbol{V_i})$.
Run h independent SDPA heads in parallel.
This yields h separate output matrices $(\boldsymbol{X_1}, \ldots, \boldsymbol{X_h})$.
Concatenate all h outputs: $Concat(\boldsymbol{X_1}, \ldots, \boldsymbol{X_h})$.
Apply a final linear projection $\boldsymbol{W^O}$.



PyTorch 中的多头注意力（MHA）

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, h):
        super().__init__()
        self.d_model = d_model # e.g., 768
        self.h = h             # e.g., 12
        self.d_k = d_model // h  # e.g., 64

        # --- Learnable Weights ---

        # Linear layers for Q, K, V (W_q, W_k, W_v)
        # We define one big linear layer for each
        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)

        # Final output projection (W_o)
        self.out_linear = nn.Linear(d_model, d_model)

    # (forward method on next slide...)
```

Multi-Head Attention (MHA) in PyTorch

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, h):
        super().__init__()
        self.d_model = d_model # e.g., 768
        self.h = h             # e.g., 12
        self.d_k = d_model // h  # e.g., 64

        # --- Learnable Weights ---

        # Linear layers for Q, K, V (W_q, W_k, W_v)
        # We define one big linear layer for each
        self.q_linear = nn.Linear(d_model, d_model)
        self.k_linear = nn.Linear(d_model, d_model)
        self.v_linear = nn.Linear(d_model, d_model)

        # Final output projection (W_o)
        self.out_linear = nn.Linear(d_model, d_model)

    # (forward method on next slide...)
```

PyTorch 中的多头注意力（MHA）

```
    def forward(self, x, mask=None):
        # x shape: [Batch, Seq_Len (T), d_model]
        batch_size = x.size(0)
        # 1. Project and split into h heads
        # Q/K/V shape: [B, T, d_model] -> [B, h, T, d_k]
        Q = self.q_linear(x).view(batch_size, -1, self.h, self.d_k).
            transpose(1,2)
        K = self.k_linear(x).view(batch_size, -1, self.h, self.d_k).
            transpose(1,2)
        V = self.v_linear(x).view(batch_size, -1, self.h, self.d_k).
            transpose(1,2)

        # 2. Run all heads in parallel (using our function)
        # new_x shape: [B, h, T, d_k]
        new_x, weights = scaled_dot_product_attention(Q, K, V, mask)

        # 3. Concat and 4. Project
        # [B, h, T, d_k] -> [B, T, h, d_k] -> [B, T, d_model]
        new_x = new_x.transpose(1, 2).contiguous().view(batch_size,
            -1, self.d_model)
        return self.out_linear(new_x)
```

- view() 不会创建新特征。
- 它只是重组张量的形状。
- 这里把 768 维向量拆成 12 个头，每个头 64 维。

Multi-Head Attention (MHA) in PyTorch

```
    def forward(self, x, mask=None):
        # x shape: [Batch, Seq_Len (T), d_model]
        batch_size = x.size(0)
        # 1. Project and split into h heads
        # Q/K/V shape: [B, T, d_model] -> [B, h, T, d_k]
        Q = self.q_linear(x).view(batch_size, -1, self.h, self.d_k).
            transpose(1,2)
        K = self.k_linear(x).view(batch_size, -1, self.h, self.d_k).
            transpose(1,2)
        V = self.v_linear(x).view(batch_size, -1, self.h, self.d_k).
            transpose(1,2)

        # 2. Run all heads in parallel (using our function)
        # new_x shape: [B, h, T, d_k]
        new_x, weights = scaled_dot_product_attention(Q, K, V, mask)

        # 3. Concat and 4. Project
        # [B, h, T, d_k] -> [B, T, h, d_k] -> [B, T, d_model]
        new_x = new_x.transpose(1, 2).contiguous().view(batch_size,
            -1, self.d_model)
        return self.out_linear(new_x)
```

- view() does not create new features.
- It only reorganizes the tensor shape.
- Here, it splits the 768-dimensional vector into 12 heads, each with 64 dimensions.

组件 4： 前馈层

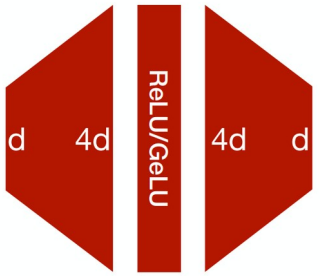
该模块位于 MHA 层之后。

- 它是一个简单的两层全连接神经网络。
- 第一层扩展维度： $X \rightarrow X'$ （使用 W_1, b_1 ），配合 ReLU 激活。

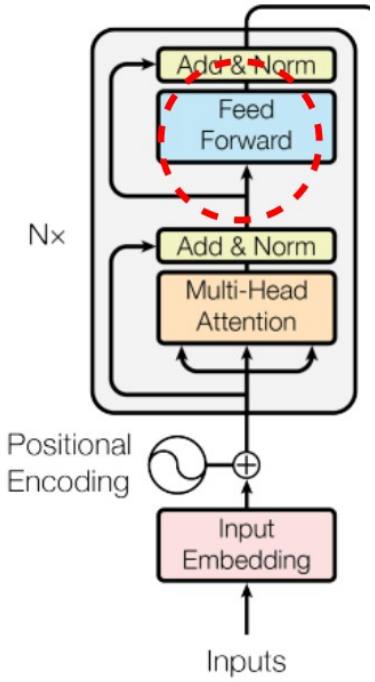
$$X' = ReLU(XW_1 + b_1)$$

- 第二层压缩回去： $X' \rightarrow X''$ （使用 W_2, b_2 ）

$$X'' = X'W_2 + b_2$$



- 例如 GPT-3 Small： $d = 768 \rightarrow 3072 \rightarrow 768$
- 权重 W_1 、 W_2 和向量 b_1 、 b_2 都是可训练参数。



Component 4: Feed-Forward Layer

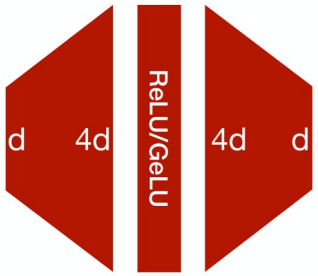
This block is applied after the MHA layer.

- It is a simple two-layer fully connected neural network.
- First layer expands dimension: $X \rightarrow X'$ (using W_1, b_1) with ReLU activation.

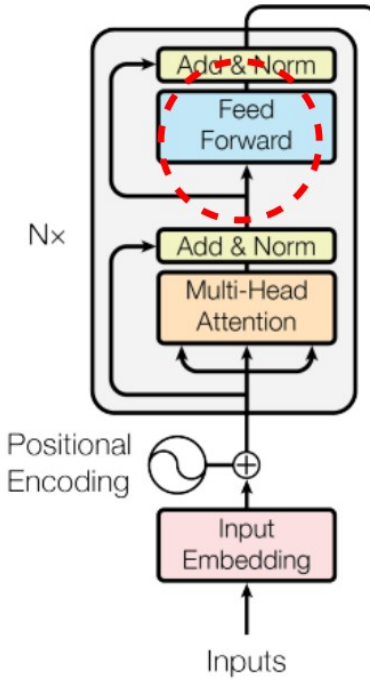
$$X' = ReLU(XW_1 + b_1)$$

- Second layer compresses back: $X' \rightarrow X''$ (using W_2, b_2)

$$X'' = X'W_2 + b_2$$



- e.g., GPT-3 Small: d = 768 → 3072 → 768
- The weights W_1, W_2 and vectors b_1, b_2 are trainable parameters.



PyTorch 中的前馈层

```
import torch
import torch.nn as nn
class FeedForward(nn.Module):
    # Implements FFN(X) = ReLU(XW1
        + b1)W2 + b2
    def __init__(self, d_model,
        d_ff):
        super().__init__()
        # W1 and b1 (e.g., 768 ->
            3072)
        self.linear_1 = nn.Linear(
            d_model, d_ff
        )
        self.relu = nn.ReLU()
        # W2 and b2 (e.g., 3072 ->
            768)
        self.linear_2 = nn.Linear(
            d_ff, d_model
        )
    def forward(self, x):
        # x shape: [Batch, Seq_Len
            , d_model]
        # (1) x -> [B, T, d_ff]
        x = self.relu(self.
            linear_1(x))
        # (2) x -> [B, T, d_model]
        x = self.linear_2(x)
        return x
# --- Example Usage ---
d_model = 768
d_ff = 3072 # 768 * 4
ffn_block = FeedForward(d_model,
    d_ff)
# Create a dummy input from MHA
dummy_x = torch.randn(
    2, 10, d_model
) # [B, T, d_model]
output = ffn_block(dummy_x)
# output.shape: [2, 10, 768]
```

Feed-Forward Layer in PyTorch

```
import torch
import torch.nn as nn
class FeedForward(nn.Module):
    # Implements FFN(X) = ReLU(XW1
        + b1)W2 + b2
    def __init__(self, d_model,
        d_ff):
        super().__init__()
        # W1 and b1 (e.g., 768 ->
            3072)
        self.linear_1 = nn.Linear(
            d_model, d_ff
        )
        self.relu = nn.ReLU()
        # W2 and b2 (e.g., 3072 ->
            768)
        self.linear_2 = nn.Linear(
            d_ff, d_model
        )
```

```
    def forward(self, x):
        # x shape: [Batch, Seq_Len
            , d_model]
        # (1) x -> [B, T, d_ff]
        x = self.relu(self.
            linear_1(x))
        # (2) x -> [B, T, d_model]
        x = self.linear_2(x)
        return x
# --- Example Usage ---
d_model = 768
d_ff = 3072 # 768 * 4
ffn_block = FeedForward(d_model,
    d_ff)
# Create a dummy input from MHA
dummy_x = torch.randn(
    2, 10, d_model
) # [B, T, d_model]
output = ffn_block(dummy_x)
# output.shape: [2, 10, 768]
```

粘合剂： Add & Norm

每个子层（MHA 或 FFN）都配合两种操作： Add 与 Norm。

残差连接（Add）： 受 ResNet 启发

- 输入被加到子层输出上： X + SubLayer(X)。
- 这条捷径让梯度更直接地流动，使深层网络更易训练。

层归一化（Norm）：

- 对每个 token，计算其 d 个特征的均值和方差，并归一化到大致均值 0、方差 1。
- 再施加可学习的缩放参数 γ 与平移参数 β，有助于稳定训练

$$\mu_j = \frac{1}{d}\sum_{k=1}^{d} x_j[k], \sigma_j^2 = \frac{1}{d}\sum_{k=1}^{d}(x_j[k] - \mu_j)^2 \qquad x_j' = \gamma \odot \left(\frac{x_j - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}\right) + \beta$$

- 现代 Transformer 常用 Pre-Norm：在子层之前做 LayerNorm。

The Glue: Add & Norm

Each sub-layer (MHA or FFN) is combined with two operations: Add and Norm.

Residual Connection (Add): Inspired from ResNet

- The input is added to the sub-layer output: X + SubLayer(X).
- This shortcut helps gradients flow more directly, making deep networks easier to train.

Layer Normalization (Norm):

- For each token, compute the mean and variance across its *d* features, and normalize them to roughly mean 0 and variance 1.
- Then apply learnable scale γ and shift β parameters, which help stabilize training

$$\mu_j = \frac{1}{d}\sum_{k=1}^{d} \boldsymbol{x_j}[k], \sigma_j^2 = \frac{1}{d}\sum_{k=1}^{d}(\boldsymbol{x_j}[k] - \mu_j)^2 \qquad \boldsymbol{x_j'} = \gamma \odot (\frac{\boldsymbol{x_j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}) + \beta$$

- Modern Transformers often use Pre-Norm: LayerNorm is applied before the sub-layer.

PyTorch 中的 Add & Pre-Norm

```
import torch.nn as nn
class ResidualConnection(nn.Module):
    # Implements Pre-Norm: X_final = X + SubLayer(LN(X))
    def __init__(self, sublayer,
                 d_model, dropout):
        super().__init__()
        self.sublayer = sublayer
        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

# --- Example (in your
# --- TransformerLayer __init__)
# self.mha = MultiHeadAttention(...)
# self.ffn = FeedForward(...)

# self.mha_res = ResidualConnection(
#     self.mha, d_model, dropout
# )
# self.ffn_res = ResidualConnection(
#     self.ffn, d_model, dropout
# )
```

Add & Pre-Norm in PyTorch

```
import torch.nn as nn
class ResidualConnection(nn.Module):
    # Implements Pre-Norm: X_final = X + SubLayer(LN(X))
    def __init__(self, sublayer,
                 d_model, dropout):
        super().__init__()
        self.sublayer = sublayer
        self.norm = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

# --- Example (in your
# --- TransformerLayer __init__)
# self.mha = MultiHeadAttention(...)
# self.ffn = FeedForward(...)

# self.mha_res = ResidualConnection(
#     self.mha, d_model, dropout
# )
# self.ffn_res = ResidualConnection(
#     self.ffn, d_model, dropout
# )
```

PyTorch 中的 Add & Pre-Norm

```
    def forward(self, x):
        # x shape: [B, T, d_model]

        # 1. Normalize first (Pre-Norm)
        x_norm = self.norm(x)

        # 2. Pass through sublayer (MHA/FFN)
        sub_output = self.sublayer(x_norm)

        # 3. Apply dropout
        sub_output = self.dropout(sub_output)

        # 4. Add residual connection
        return x + sub_output

# --- Example (in your
# --- TransformerLayer forward)
# x = ...
# x = self.mha_res(x)
# x = self.ffn_res(x)
# return x
```

Add & Pre-Norm in PyTorch

```
    def forward(self, x):
        # x shape: [B, T, d_model]

        # 1. Normalize first (Pre-Norm)
        x_norm = self.norm(x)

        # 2. Pass through sublayer (MHA/FFN)
        sub_output = self.sublayer(x_norm)

        # 3. Apply dropout
        sub_output = self.dropout(sub_output)

        # 4. Add residual connection
        return x + sub_output

# --- Example (in your
# --- TransformerLayer forward)
# x = ...
# x = self.mha_res(x)
# x = self.ffn_res(x)
# return x
```

组装起来： Transformer 模块

- 我们已经看过构建 Transformer 层所需的全部“乐高积木”（组件）。
- 单个 Transformer 模块（或层）按固定顺序组合这些组件。

Input: X_{in} (from previous layer)

Sub-layer 1 (MHA):

- $X_{norm1} = LayerNorm(X_{in})$
- $X_{mha} = MultiHeadAttention(X_{norm1})$
- $X_{mid} = X_{in} + X_{mha}$ (1st Residual Connection)

Sub-layer 2 (FFN):

- $X_{norm2} = LayerNorm(X_{mid})$
- $X_{ffn} = FeedForward(X_{norm2})$
- $X_{out} = X_{mid} + X_{ffn}$ (2nd Residual Connection)

Output: X_{out} (to next layer)

- 再把整个模块堆叠 N 次（如 N = 12）构成模型

Putting It All Together: A Transformer Block

- We have now seen all the “LEGO bricks” (components) needed to build a Transformer layer.
- A single Transformer Block (or Layer) combines these components in a fixed order.

Input: X_{in} (from previous layer)

Sub-layer 1 (MHA):

- $X_{norm1} = LayerNorm(X_{in})$
- $X_{mha} = MultiHeadAttention(X_{norm1})$
- $X_{mid} = X_{in} + X_{mha}$ (1st Residual Connection)

Sub-layer 2 (FFN):

- $X_{norm2} = LayerNorm(X_{mid})$
- $X_{ffn} = FeedForward(X_{norm2})$
- $X_{out} = X_{mid} + X_{ffn}$ (2nd Residual Connection)

Output: X_{out} (to next layer)

- This entire block is then stacked N times (e.g., N = 12) to build the model

PyTorch 中的 Transformer 模块

```
class TransformerBlock(nn.Module):
    """
    Implements one full Transformer Block (MHA -> Add&Norm -> FFN ->
        Add&Norm) using the Pre-Norm style.
    """
    def __init__(self, d_model, h, d_ff, dropout_p):
        super().__init__()
        # 1. Multi-Head Attention sub-layer
        self.mha = MultiHeadAttention(d_model, h)
        # 2. Feed-Forward sub-layer
        self.ffn = FeedForward(d_model, d_ff)
        # 3. Two ResidualConnection wrappers
        self.res_conn_1 = ResidualConnection(
            sublayer_module = self.mha,
            d_model = d_model,
            dropout_p = dropout_p
        )
        self.res_conn_2 = ResidualConnection(
            sublayer_module = self.ffn,
            d_model = d_model,
            dropout_p = dropout_p
        )
```

Transformer Block in PyTorch

```
class TransformerBlock(nn.Module):
    """
    Implements one full Transformer Block (MHA -> Add&Norm -> FFN ->
        Add&Norm) using the Pre-Norm style.
    """
    def __init__(self, d_model, h, d_ff, dropout_p):
        super().__init__()
        # 1. Multi-Head Attention sub-layer
        self.mha = MultiHeadAttention(d_model, h)
        # 2. Feed-Forward sub-layer
        self.ffn = FeedForward(d_model, d_ff)
        # 3. Two ResidualConnection wrappers
        self.res_conn_1 = ResidualConnection(
            sublayer_module = self.mha,
            d_model = d_model,
            dropout_p = dropout_p
        )
        self.res_conn_2 = ResidualConnection(
            sublayer_module = self.ffn,
            d_model = d_model,
            dropout_p = dropout_p
        )
```

PyTorch 中的 Transformer 模块

```
    def forward(self, x, mask=None):
        # x shape: [B, T, d_model]
        # 1. Apply MHA block:
        # X_mid = X_in + MHA(LN(X_in))
        # We must pass the mask to MHA
        x = self.res_conn_1(x, mask=mask)
        # 2. Apply FFN block:
        # X_out = X_mid + FFN(LN(X_mid))
        # No mask needed for FFN
        x = self.res_conn_2(x)
        return x
# --- Full Model (Conceptual) ---
# self.embedding = ...
# self.rope = ...
# self.transformer_blocks = nn.ModuleList([
#     TransformerBlock(d_model, h, d_ff, dropout)
#     for _ in range(N_LAYERS)])
# x = self.embedding(input_ids)
# for block in self.transformer_blocks:
#     x = block(x, mask=my_mask)
```

Transformer Block in PyTorch

```
    def forward(self, x, mask=None):
        # x shape: [B, T, d_model]
        # 1. Apply MHA block:
        # X_mid = X_in + MHA(LN(X_in))
        # We must pass the mask to MHA
        x = self.res_conn_1(x, mask=mask)
        # 2. Apply FFN block:
        # X_out = X_mid + FFN(LN(X_mid))
        # No mask needed for FFN
        x = self.res_conn_2(x)
        return x
# --- Full Model (Conceptual) ---
# self.embedding = ...
# self.rope = ...
# self.transformer_blocks = nn.ModuleList([
#     TransformerBlock(d_model, h, d_ff, dropout)
#     for _ in range(N_LAYERS)])
# x = self.embedding(input_ids)
# for block in self.transformer_blocks:
#     x = block(x, mask=my_mask)
```

有了 Transformer 模块，下一步是什么?

- 最初的 Transformer 是为序列到序列任务（如翻译）设计的。

- 为此它使用两个独立的堆栈，各自采用该模块的专门版本：
  - **编码器**堆栈：“理解”输入序列。
  - **解码器**堆栈：“生成”输出序列。

We Have a Transformer Block, What's next?

- The original Transformer was designed for sequence-to-sequence tasks (like translation).

- To do this, it uses two separate stacks, each using a specialized version of our block:
  - An **Encoder** Stack: to "understand" the input sequence.
  - A **Decoder** Stack: to "generate" the output sequence.

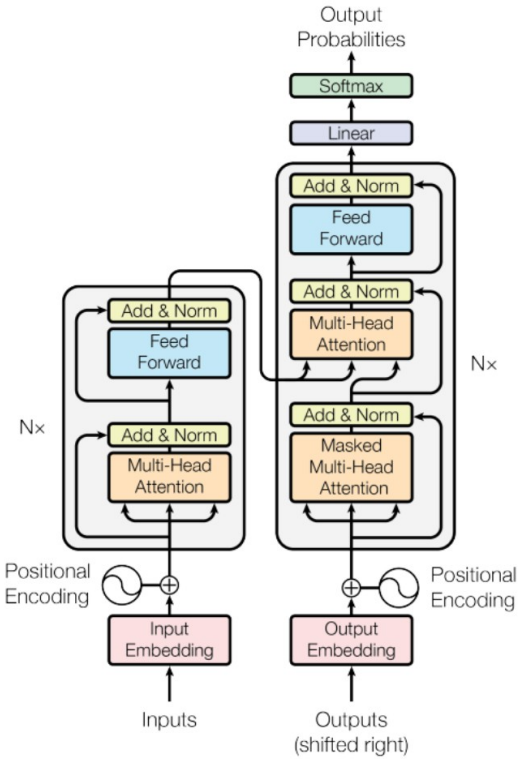
编码器堆栈 vs. 解码器堆栈

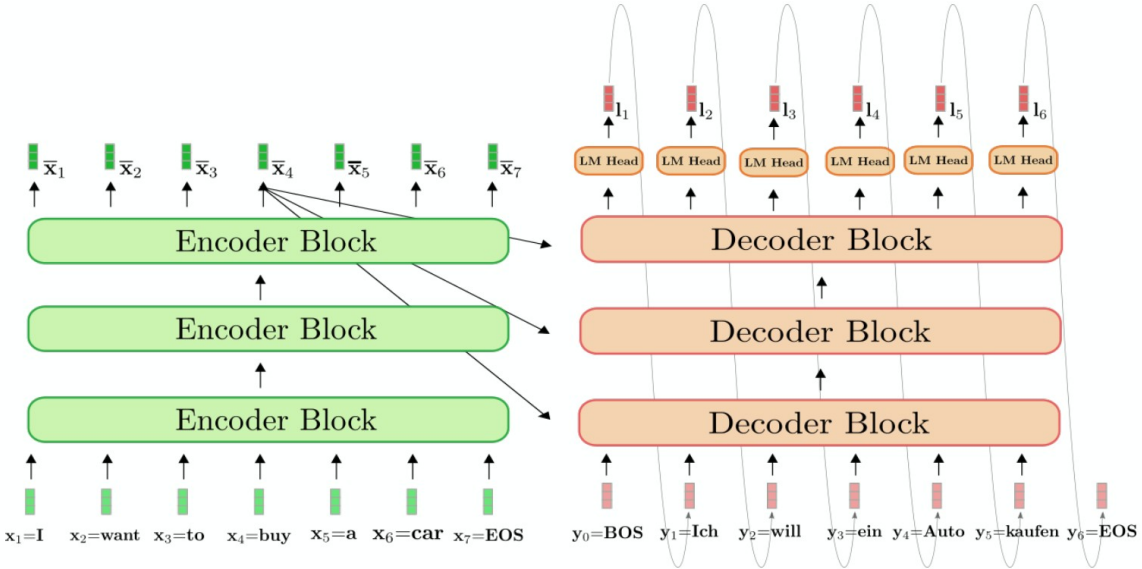
编码器堆栈：包含多头注意力层和前馈层。

- 双向：每个位置都能看到其他所有位置

解码器堆栈：包含掩码 MHA 、交叉注意力和前馈层。

- 自回归：每次预测只能使用之前的位置





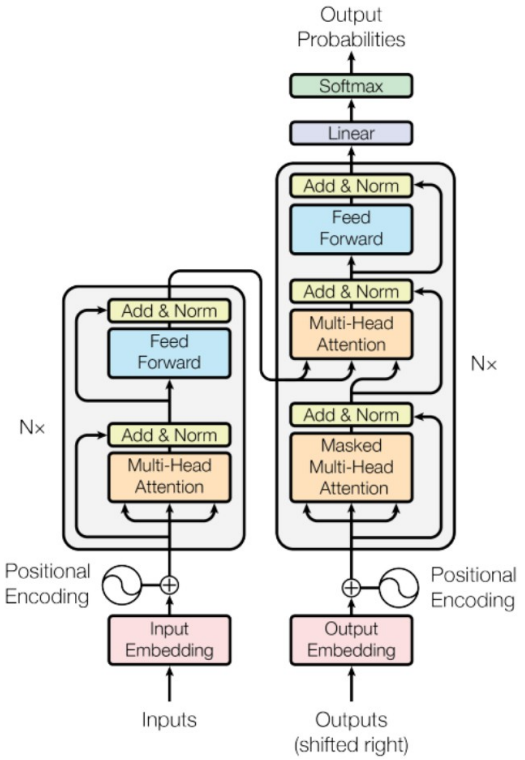
Encoder vs. Decoder Stacks

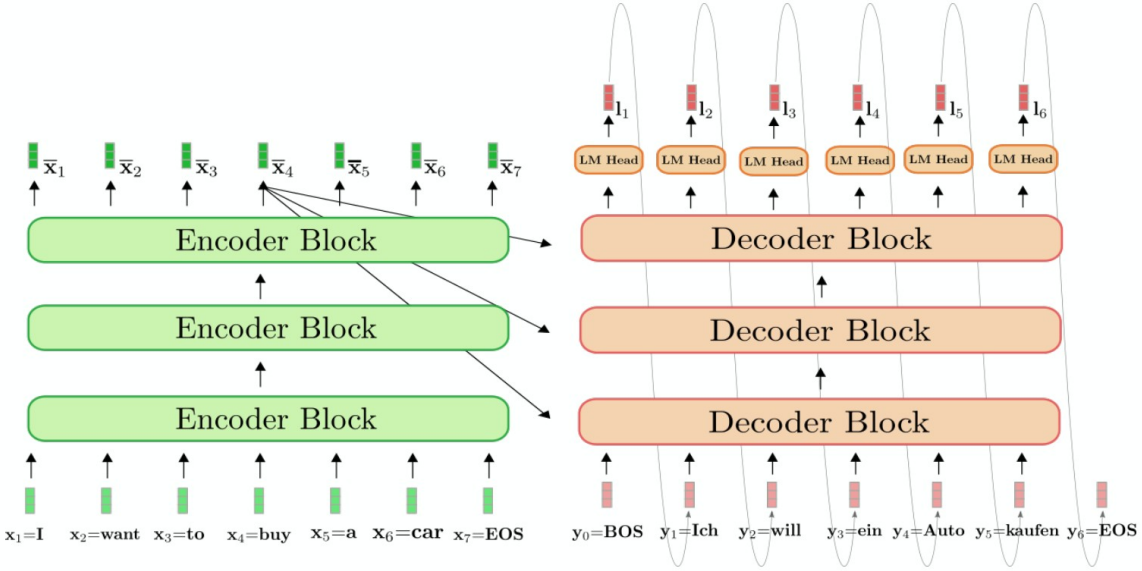
Encoder Stack: with Multi-Head Attention layers and Feed-Forward layers.

- Bidirectional: Each position can see all other positions

Decoder Stack: with Masked MHA, Cross-Attention, and Feed-Forward layers.

- Autoregressive: Each prediction can only use previous positions





解码器：掩码多头注意力

目的：为实现自回归生成，解码器不能关注未来的目标位置。

工作原理

- 在 softmax 之前对注意力得分施加掩码。
- 把未来位置设为 $-\infty$
- softmax 之后，这些未来位置的概率为 0 。

$$S_{i_masked} = S_i + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

Decoder: Masked Multi-Head Attention

Purpose: For autoregressive generation, the decoder must not attend to future target positions.

How it works

- Apply the mask to the attention scores before softmax.
- Set future positions to $-\infty$
- After softmax, those future positions get probability 0.

$$S_{i_masked} = S_i + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

PyTorch 中的因果（前瞻）掩码

```
import torch
# 1. Create an upper-triangular matrix of 1s (True)
# torch.triu(..., diagonal=1) -> Upper triangle, k=1
# [[0., 1., 1., 1., 1.], [0., 0., 1., 1., 1.],
#  [0., 0., 0., 1., 1.], [0., 0., 0., 0., 1.],
#  [0., 0., 0., 0., 0.]]
causal_mask_bool = torch.triu(torch.ones(seq_len, seq_len), diagonal
    =1).bool()
print("Causal Mask (True where we fill):\n", causal_mask_bool)

# 2. Add Batch and Head dimensions for broadcasting
final_mask = causal_mask_bool.unsqueeze(0).unsqueeze(0)
print("\nFinal mask shape for SDPA:", final_mask.shape)

# 3. Test in SDPA function
dummy_q = torch.randn(1, 1, seq_len, 64)
dummy_k = torch.randn(1, 1, seq_len, 64)
dummy_v = torch.randn(1, 1, seq_len, 64)
# Pass the mask where True means 'fill'
output, weights = scaled_dot_product_attention(dummy_q, dummy_k,
    dummy_v, mask=final_mask)
```

Causal (Look-Ahead) Mask in PyTorch

```
import torch
# 1. Create an upper-triangular matrix of 1s (True)
# torch.triu(..., diagonal=1) -> Upper triangle, k=1
# [[0., 1., 1., 1., 1.], [0., 0., 1., 1., 1.],
#  [0., 0., 0., 1., 1.], [0., 0., 0., 0., 1.],
#  [0., 0., 0., 0., 0.]]
causal_mask_bool = torch.triu(torch.ones(seq_len, seq_len), diagonal
    =1).bool()
print("Causal Mask (True where we fill):\n", causal_mask_bool)

# 2. Add Batch and Head dimensions for broadcasting
final_mask = causal_mask_bool.unsqueeze(0).unsqueeze(0)
print("\nFinal mask shape for SDPA:", final_mask.shape)

# 3. Test in SDPA function
dummy_q = torch.randn(1, 1, seq_len, 64)
dummy_k = torch.randn(1, 1, seq_len, 64)
dummy_v = torch.randn(1, 1, seq_len, 64)
# Pass the mask where True means 'fill'
output, weights = scaled_dot_product_attention(dummy_q, dummy_k,
    dummy_v, mask=final_mask)
```

解码器：交叉注意力

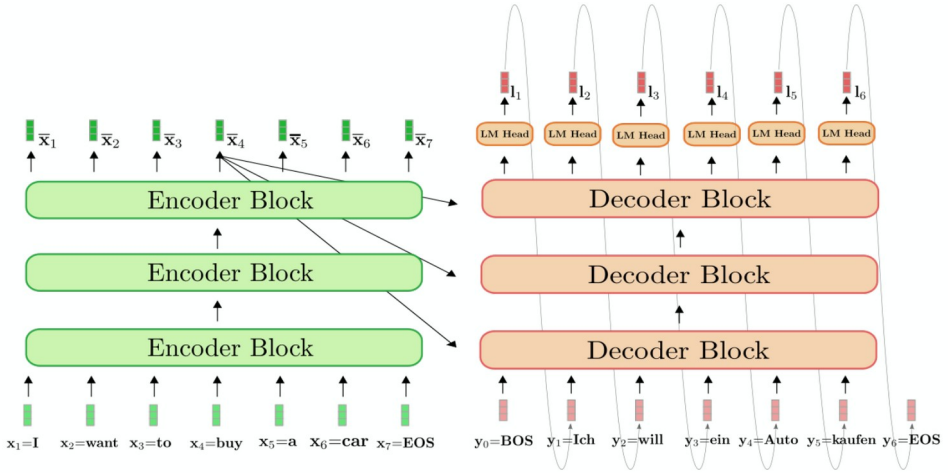
这是解码器从编码器获取信息的方式。
它让负责生成的（解码器）堆栈能够“查看”输入（编码器）堆栈。

Self-Attention (what we've seen):

- ***Q***, ***K***, ***V*** all come from the **same source** (the previous layer's output).
- 'output = mha(x, x, x)'
- "How should tokens in ***X*** attend to other tokens in ***X***?"

Cross-Attention (new):

- ***Q*** comes from the **Decoder**.
- ***K*** and ***V*** come from the **Encoder**.
- 'output = mha(x_decoder, encoder_output, encoder_output)'
- "How should Decoder tokens attend to Encoder tokens?"



Decoder: Cross-Attention

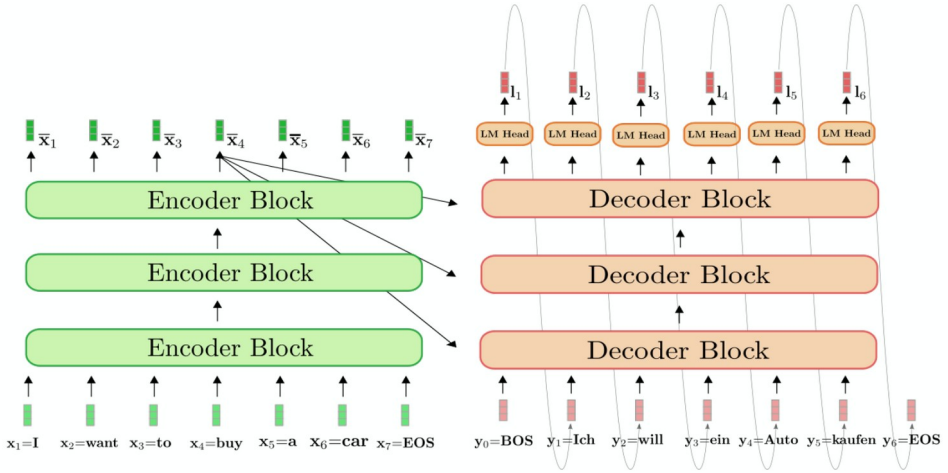
This is the way the Decoder gets information from the Encoder.
It allows the generating (Decoder) stack to “look at” the input (Encoder) stack.

Self-Attention (what we’ve seen):

- ***Q***, ***K***, ***V*** all come from the **same source** (the previous layer’s output).
- ‘output = mha(x, x, x)’
- "How should tokens in ***X*** attend to other tokens in ***X***?"

Cross-Attention (new):

- ***Q*** comes from the **Decoder**.
- ***K*** and ***V*** come from the **Encoder**.
- ‘output = mha(x_decoder, encoder_output, encoder_output)’
- “How should Decoder tokens attend to Encoder tokens?”



PyTorch 中的解码器

```
class DecoderBlock(nn.Module):
    """
    Implements one full Decoder Block
    (Masked MHA -> Add&Norm -> Cross MHA -> Add&Norm -> FFN -> Add&
        Norm)
    """
    def __init__(self, d_model, h, d_ff, dropout_p):
        super().__init__()
        # 1. Masked Multi-Head Attention (Self-Attention)
        self.masked_mha = MultiHeadAttention(d_model, h)
        # 2. Cross-Attention (MHA)
        self.cross_mha = MultiHeadAttention(d_model, h)
        # 3. Feed-Forward
        self.ffn = FeedForward(d_model, d_ff)

        # 4. Three ResidualConnection wrappers
        self.res_conn_1 = ResidualConnection(self.masked_mha, d_model,
             dropout_p)
        self.res_conn_2 = ResidualConnection(self.cross_mha, d_model,
            dropout_p)
        self.res_conn_3 = ResidualConnection(self.ffn, d_model,
            dropout_p)
```

Decoder in PyTorch

```
class DecoderBlock(nn.Module):
    """
    Implements one full Decoder Block
    (Masked MHA -> Add&Norm -> Cross MHA -> Add&Norm -> FFN -> Add&
        Norm)
    """
    def __init__(self, d_model, h, d_ff, dropout_p):
        super().__init__()
        # 1. Masked Multi-Head Attention (Self-Attention)
        self.masked_mha = MultiHeadAttention(d_model, h)
        # 2. Cross-Attention (MHA)
        self.cross_mha = MultiHeadAttention(d_model, h)
        # 3. Feed-Forward
        self.ffn = FeedForward(d_model, d_ff)

        # 4. Three ResidualConnection wrappers
        self.res_conn_1 = ResidualConnection(self.masked_mha, d_model,
             dropout_p)
        self.res_conn_2 = ResidualConnection(self.cross_mha, d_model,
            dropout_p)
        self.res_conn_3 = ResidualConnection(self.ffn, d_model,
            dropout_p)
```

PyTorch 中的解码器

```
    def forward(self, x, encoder_output, causal_mask, padding_mask=
        None):
        # x shape: [B, Target_T, D], encoder_output shape: [B,
            Source_T, D]

        # 1. Apply Masked MHA block (Self-Attention)
        # Q, K, V all come from x
        # We must use the causal (look-ahead) mask
        x = self.res_conn_1(
            x, x_k=x, x_v=x, mask=causal_mask
        )
        # 2. Apply Cross-Attention block
        # Q comes from x (decoder)
        # K, V come from encoder_output
        x = self.res_conn_2(x, x_k=encoder_output, x_v=encoder_output,
             mask=padding_mask # (Masks <pad> tokens in encoder)
        )

        # 3. Apply FFN block
        x = self.res_conn_3(x)
        return x
```

Decoder in PyTorch

```
    def forward(self, x, encoder_output, causal_mask, padding_mask=
        None):
        # x shape: [B, Target_T, D], encoder_output shape: [B,
            Source_T, D]

        # 1. Apply Masked MHA block (Self-Attention)
        # Q, K, V all come from x
        # We must use the causal (look-ahead) mask
        x = self.res_conn_1(
            x, x_k=x, x_v=x, mask=causal_mask
        )
        # 2. Apply Cross-Attention block
        # Q comes from x (decoder)
        # K, V come from encoder_output
        x = self.res_conn_2(x, x_k=encoder_output, x_v=encoder_output,
             mask=padding_mask # (Masks <pad> tokens in encoder)
        )

        # 3. Apply FFN block
        x = self.res_conn_3(x)
        return x
```

解码器：最终输出分布（GPT 风格）

对 GPT 这类仅解码器模型，只保留解码器堆栈。经过 N 个解码器层后，得到最终的隐藏表示 $X \in R^{t \times d}$ 。

如何预测下一个 token？

- GPT 取最后一个位置的隐藏表示 x_t 。
- 用初始嵌入矩阵 M 把该向量投影到词表空间，计算出“ logits ”

$$z_t = x_t^\top M \in \mathbb{R}^{|\mathcal{Y}|}$$

- logits z_t 经 Softmax 函数转换为概率分布。

$$P(y|context) = \frac{exp(z_t[y])}{\sum_{y' \in \mathcal{Y}} exp(z_t[y'])}$$

Decoder: Final Output Distribution (GPT-style)

For decoder-only models such as GPT, we keep only the Decoder stack. After N decoder layers, we have a final set of hidden representations $X \in R^{txd}$.

How to predict the next token?

- GPT takes the hidden representation at the last position, x_t.
- It computes "logits" by projecting this vector into the vocabulary space using the initial embedding matrix M

$$z_t = x_t^\top M \in \mathbb{R}^{|\mathcal{Y}|}$$

- The logits z_t are converted to a probability distribution using the Softmax function.

$$P(y|context) = \frac{exp(z_t[y])}{\sum_{y' \in \mathcal{Y}} exp(z_t[y'])}$$

PyTorch 中的解码器最终输出

```
class GPTModel(nn.Module):
    def __init__(self, vocab_size, d_model, n_layers, h, ...):
        super().__init__()
        # 1. Token Embedding (Matrix M)
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        # 2. Stack of N Decoder-Only Blocks
        # (Note: A GPT block is a DecoderBlock *without* the Cross-
            Attention sub-layer)
        self.blocks = nn.ModuleList([
            GPTDecoderBlock(d_model, h, ...)
            for _ in range(n_layers)
        ])
        # 3. Final Layer Normalization
        self.final_norm = nn.LayerNorm(d_model)
        # 4. Final Linear Layer (Projection to Vocab)
        self.final_linear = nn.Linear(d_model, vocab_size)
        # 5. !!! WEIGHT TYING !!!
        # The weights of the embedding layer (M) and the final linear
            layer are shared.
        self.final_linear.weight = self.token_embedding.weight
```

Decoder Final Output in PyTorch

```
class GPTModel(nn.Module):
    def __init__(self, vocab_size, d_model, n_layers, h, ...):
        super().__init__()
        # 1. Token Embedding (Matrix M)
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        # 2. Stack of N Decoder-Only Blocks
        # (Note: A GPT block is a DecoderBlock *without* the Cross-
            Attention sub-layer)
        self.blocks = nn.ModuleList([
            GPTDecoderBlock(d_model, h, ...)
            for _ in range(n_layers)
        ])
        # 3. Final Layer Normalization
        self.final_norm = nn.LayerNorm(d_model)
        # 4. Final Linear Layer (Projection to Vocab)
        self.final_linear = nn.Linear(d_model, vocab_size)
        # 5. !!! WEIGHT TYING !!!
        # The weights of the embedding layer (M) and the final linear
            layer are shared.
        self.final_linear.weight = self.token_embedding.weight
```

PyTorch 中的解码器最终输出

```
def forward(self, input_ids, targets=None):
    # input_ids: [Batch, Seq_Len]
    # 1. Get Token Embeddings, x shape: [B, T, D]
    x = self.token_embedding(input_ids)
    # 2. Pass through all Decoder Blocks
    for block in self.blocks:
        x = block(x)
    # 3. Final Norm
    x = self.final_norm(x)
    # 4. Project to Vocab Space (Logits)
    logits = self.final_linear(x)
    # logits shape: [B, T, Vocab_Size]
    # 5. Calculate Loss (During Training)
    if targets is not None:
        # Flatten for CrossEntropyLoss
        # (B, T, V) -> (B*T, V); (B, T) -> (B*T)
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)),
            targets.view(-1))
    else:
        loss = None
    return logits, loss
```

Decoder Final Output in PyTorch

```
def forward(self, input_ids, targets=None):
    # input_ids: [Batch, Seq_Len]
    # 1. Get Token Embeddings, x shape: [B, T, D]
    x = self.token_embedding(input_ids)
    # 2. Pass through all Decoder Blocks
    for block in self.blocks:
        x = block(x)
    # 3. Final Norm
    x = self.final_norm(x)
    # 4. Project to Vocab Space (Logits)
    logits = self.final_linear(x)
    # logits shape: [B, T, Vocab_Size]
    # 5. Calculate Loss (During Training)
    if targets is not None:
        # Flatten for CrossEntropyLoss
        # (B, T, V) -> (B*T, V); (B, T) -> (B*T)
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)),
            targets.view(-1))
    else:
        loss = None
    return logits, loss
```

完整的 Transformer

编码器

```
# (We assume EncoderBlock is already defined)

class Encoder(nn.Module):
    def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
        super().__init__()
        # Create N identical EncoderBlocks
        self.layers = nn.ModuleList([
            EncoderBlock(d_model, h, d_ff, dropout_p)
            for _ in range(n_layers)
        ])
        # Final normalization
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, mask=None):
        # x shape: [B, T_src, D]
        # Pass input through all N layers
        for layer in self.layers:
            x = layer(x, mask=mask)

        # Final normalization
        return self.norm(x)
```

Full ‘Transformer’

Encoder

```
# (We assume EncoderBlock is already defined)

class Encoder(nn.Module):
    def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
        super().__init__()
        # Create N identical EncoderBlocks
        self.layers = nn.ModuleList([
            EncoderBlock(d_model, h, d_ff, dropout_p)
            for _ in range(n_layers)
        ])
        # Final normalization
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, mask=None):
        # x shape: [B, T_src, D]
        # Pass input through all N layers
        for layer in self.layers:
            x = layer(x, mask=mask)

        # Final normalization
        return self.norm(x)
```

完整的 Transformer

解码器

```
# (We assume DecoderBlock is already defined)
class Decoder(nn.Module):
    def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
        super().__init__()
        # Create N identical DecoderBlocks
        self.layers = nn.ModuleList([
            DecoderBlock(d_model, h, d_ff, dropout_p)
            for _ in range(n_layers)
        ])
        # Final normalization
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, enc_output, causal_mask, padding_mask):
        # x shape: [B, T_tgt, D]
        # Pass input through all N layers
        for layer in self.layers:
            x = layer(x, encoder_output=enc_output, causal_mask=
                causal_mask, padding_mask=padding_mask)
        # Final normalization
        return self.norm(x)
```

Full ‘Transformer’

Decoder

```
# (We assume DecoderBlock is already defined)
class Decoder(nn.Module):
    def __init__(self, n_layers, d_model, h, d_ff, dropout_p):
        super().__init__()
        # Create N identical DecoderBlocks
        self.layers = nn.ModuleList([
            DecoderBlock(d_model, h, d_ff, dropout_p)
            for _ in range(n_layers)
        ])
        # Final normalization
        self.norm = nn.LayerNorm(d_model)

    def forward(self, x, enc_output, causal_mask, padding_mask):
        # x shape: [B, T_tgt, D]
        # Pass input through all N layers
        for layer in self.layers:
            x = layer(x, encoder_output=enc_output, causal_mask=
                causal_mask, padding_mask=padding_mask)
        # Final normalization
        return self.norm(x)
```

完整的 Transformer

Transformer

```
class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model,
        n_layers, h, d_ff, dropout_p, max_len):
        super().__init__()
        # --- Embeddings ---
        # (We use nn.Embedding for absolute pos,
        #  but RoPE would be inside MHA)
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, dropout_p,
            max_len)

        # --- Encoder & Decoder Stacks ---
        self.encoder = Encoder(n_layers, d_model, h, d_ff, dropout_p)
        self.decoder = Decoder(n_layers, d_model, h, d_ff, dropout_p)

        # --- Final Output Layer ---
        self.final_linear = nn.Linear(d_model, vocab_size)

        # --- (Optional) Weight Tying ---
        # self.final_linear.weight = self.token_embedding.weight
```

Full ‘Transformer’

Transformer

```
class Transformer(nn.Module):
    def __init__(self, vocab_size, d_model,
        n_layers, h, d_ff, dropout_p, max_len):
        super().__init__()
        # --- Embeddings ---
        # (We use nn.Embedding for absolute pos,
        #  but RoPE would be inside MHA)
        self.token_embedding = nn.Embedding(vocab_size, d_model)
        self.pos_encoding = PositionalEncoding(d_model, dropout_p,
            max_len)

        # --- Encoder & Decoder Stacks ---
        self.encoder = Encoder(n_layers, d_model, h, d_ff, dropout_p)
        self.decoder = Decoder(n_layers, d_model, h, d_ff, dropout_p)

        # --- Final Output Layer ---
        self.final_linear = nn.Linear(d_model, vocab_size)

        # --- (Optional) Weight Tying ---
        # self.final_linear.weight = self.token_embedding.weight
```

完整的 Transformer

Transformer

```
def forward(self, src_ids, tgt_ids, src_mask, tgt_mask):
    # src_ids: [B, T_src]; tgt_ids: [B, T_tgt]
    # src_mask: [B, 1, 1, T_src] (for padding)
    # tgt_mask: [B, 1, T_tgt, T_tgt] (causal + padding)

    # 1. Encode the source sequence
    # src_embed: [B, T_src, D]
    src_embed = self.pos_encoding(self.token_embedding(src_ids))
    # enc_output: [B, T_src, D]
    enc_output = self.encoder(src_embed, mask=src_mask)

    # 2. Decode the target sequence
    # tgt_embed: [B, T_tgt, D]
    tgt_embed = self.pos_encoding(self.token_embedding(tgt_ids))
    # dec_output: [B, T_tgt, D]
    dec_output = self.decoder(tgt_embed, enc_output, causal_mask=
        tgt_mask, padding_mask=src_mask)
    # 3. Final projection to logits
    # logits: [B, T_tgt, vocab_size]
    logits = self.final_linear(dec_output)
    return logits
```

Full ‘Transformer’

Transformer

```
def forward(self, src_ids, tgt_ids, src_mask, tgt_mask):
    # src_ids: [B, T_src]; tgt_ids: [B, T_tgt]
    # src_mask: [B, 1, 1, T_src] (for padding)
    # tgt_mask: [B, 1, T_tgt, T_tgt] (causal + padding)

    # 1. Encode the source sequence
    # src_embed: [B, T_src, D]
    src_embed = self.pos_encoding(self.token_embedding(src_ids))
    # enc_output: [B, T_src, D]
    enc_output = self.encoder(src_embed, mask=src_mask)

    # 2. Decode the target sequence
    # tgt_embed: [B, T_tgt, D]
    tgt_embed = self.pos_encoding(self.token_embedding(tgt_ids))
    # dec_output: [B, T_tgt, D]
    dec_output = self.decoder(tgt_embed, enc_output, causal_mask=
        tgt_mask, padding_mask=src_mask)
    # 3. Final projection to logits
    # logits: [B, T_tgt, vocab_size]
    logits = self.final_linear(dec_output)
    return logits
```

Transformer 为何胜出?

- 它实现了大规模并行与可扩展性。

- 它是大多数现代语言模型的架构基础:
  - T5 、 BART （编码器 - 解码器）
  - BERT （仅编码器）
  - GPT 、 DeepSeek 、 Qwen 、 Llama （仅解码器）

Why did Transformer win ?

- It enabled massive parallelization and scalability.

- It is the architectural foundation for most modern language models:
  - T5, BART (Encoder-Decoder)
  - BERT (Encoder-only)
  - GPT, DeepSeek, Qwen, Llama (Decoder-only)

仅解码器模型的兴起

- 在当前的大语言模型时代，仅解码器已成为通用语言模型的主流架构。
  - 示例： OpenAI 的 GPT 系列就是一叠解码器模块（如前所见，但去掉了交叉注意力部分），以“下一个 token 预测”为核心任务训练。

- 然而许多前沿大模型是专有的或主要通过 API 提供，我们无法直接查看其权重或自行运行。

The Rise of Decoder-Only Models

- In the current era of LLMs, Decoder-Only has become the dominant architecture for general-purpose language models.
  - Example: GPT series from OpenAI are simply a stack of Decoder blocks (like we've seen, but without the cross-attention part) trained on the core task of Next-Token Prediction.

- However, many frontier LLMs are proprietary or available mainly through APIs, so we cannot directly inspect their weights or run the models ourselves.

开源生态

这也催生了庞大的开源生态，使强大的模型更易获取。

主要开源模型：

- DeepSeek （来自 DeepSeek AI ）
- Qwen （来自阿里巴巴）
- Llama （来自 Meta ）
- ...

如何使用这些模型?

Open-Source Ecosystem

This has also led to a large open-source ecosystem, making powerful models more accessible.

Key Open-Source Models:

- DeepSeek (from DeepSeek AI)
- Qwen (from Alibaba)
- Llama (from Meta)
- …

How can we use these models?

Python 编程与人工智能

感谢聆听！

Python Programming with Artificial Intelligence

Thank you for listening!

